

# 1 Einleitung

## 1.1 Motivation

Die Erlebniswelt von Zotter Schokolade zählt mit ihren über 300.000 jährlichen Besuchern (Stand 2025) zu den Top-3 der steirischen Sehenswürdigkeiten mit Bezahl-Eintritt [steiermark\_ausflugsz. Diese enormen Besucherströme stellen eine nicht unbedeutende organisatorische Herausforderung für Mitarbeiter und Management dar. Trotz der hohen Frequenz an Besuchern blieb der Parkplatz aus datentechnischer Sicht einer „Black Box“, da keine automatisierte Erfassung oder Auswertung von Fahrzeugbewegungen existierte. Informationen über die tatsächliche Auslastung, Stoßzeiten oder die Verweildauer basierten auf subjektiven Beobachtungen statt auf Daten. Dies erschwerte sowohl die operative Ressourcenplanung als auch strategische Entscheidungen erheblich.

Diese Diplomarbeit zielt darauf ab, diese Lücke zu schließen, indem ein modernes Kennzeichenerkennungssystem (ALPR) eingeführt wird, um Kennzeichen und Fahrzeugdaten zu erfassen, diese unter anderem auf Herkunft zu analysieren und anonymisiert zu speichern. Auf diese Daten aufbauend, können Analysen, grafische Dashboards mit den wichtigsten Informationen und langfristige Vorhersagen getroffen werden. Ziel ist es, mit diesen Möglichkeiten der Datensammlung und Analyse die strategische Planung, das Marketing und die Parkplatz-Logistik zu unterstützen.

## 1.2 Zielsetzung

Das Ziel dieser Diplomarbeit ist es, mit Daten des Besucherparkplatzes der Zotter Schokolade GmbH Analysen und Visualisierungen zu bieten, mit deren Hilfe bessere Entscheidungen in den Bereichen Betriebsführung, Marketing und Ressourcenplanung ermöglicht werden.

Das entwickelte System soll:

- Transparenz durch Echtzeit-Einblicke in Parkplatzauslastung, Besucherfrequenz und Stoßzeiten liefern
- Die Ressourcenplanung durch eine Datengrundlage für die strategische Planung unterstützen
- Verbesserung des zielgerichteten Marketings durch die Bereitstellung von Herkunftsanalysen
- Basis für die GHG (Greenhouse Gas Protocol) Scope 3 Emissionsberechnung des Besucheraufkommens liefern [**ghg\_scope3**]
- Datenschutz und DSGVO-Konformität durch Anonymisierung aller personenbezogenen Daten gewährleisten

Das System soll nicht:

- Individuelle Besucher identifizieren oder tracken
- Bewegungsprofile einzelner Fahrzeuge über den Parkplatzkontext hinaus erstellen
- Personenbezogene Daten dauerhaft speichern

Durch die Abgrenzung dieser Ziele wird sichergestellt, dass das System nur als Werkzeug für betriebliche Optimierung dient, ohne dabei die Privatsphäre der Besucher zu gefährden.

## 1.3 Aufgabenstellung

Um diesen Anforderungen gerecht zu werden, wurden diese in folgende Teilaufgaben aufgeteilt:

- Installation bzw. Montage und Konfiguration der erforderlichen Hardware
- Implementierung eines ALPR-Systems (Automatic License Plate Recognition) zur Kennzeichenerkennung
- Erkennung von Ein- und Ausfahrten mit Zeitstempeln
- Auswertung der Kennzeichen nach Herkunftsland und Region mit der Erkennung von Fahrzeugtypen (PKW, LKW, etc.)
- DSGVO-konforme Anonymisierung und Speicherung der erfassten Daten
- Erstellung eines Scripts für die automatische Datenbank-Initialisierung
- Entwicklung einer automatisierten Backup- und Wiederherstellungslösung
- Implementierung eines Dashboards zur Visualisierung der Daten
- Integration eines E-Mail-Benachrichtigungssystems
- Bereitstellung einer Entwickler-Dokumentation für zukünftige Wartung und Erweiterungen

## 1.4 Aufbau der Arbeit

Diese Diplomarbeit gliedert sich in Hauptkapitel, die den Entwicklungsprozess des Systems von den theoretischen Grundlagen bis zur Implementierung nachvollziehbar machen sollen. Das nächste Kapitel befasst sich mit den theoretischen Grundlagen. Die eingesetzten Technologien (Microservices-Architektur, Containerisierung und ALPR-Technologie) werden näher beleuchtet und deren Relevanz für das Projekt begründet.

In Kapitel 3 werden der Prozess der Hardwareauswahl, die eingesetzte Systemumgebung inklusive der getroffenen Entscheidungen bezüglich des Technologie-Stacks sowie die lokalen Standortgegebenheiten beschrieben. Es werden die spezifischen Anforderungen an Kamera und Server unter Berücksichtigung von Umgebungsbedingungen und Vorgaben der Betreuerfirma evaluiert und die getroffenen Entscheidungen argumentiert.

Kapitel 4 präsentiert Systementwurf und Architektur. Das Gesamtkonzept dieser sowie der Aufbau und Verantwortlichkeiten der Teilkomponenten werden dargelegt. Ebenfalls wird das Datenbankdesign und das allgemeine Sicherheits- bzw. Datenschutzkonzept beleuchtet.

Das darauffolgende Kapitel dokumentiert die Implementierung der einzelnen Services. Besonderes Augenmerk liegt hier auf der Integration der externen Komponenten wie Kamera-daten oder der Verarbeitung der Bilddaten und des Datenflusses bei der Datenbeschaffung und -verarbeitung.

Das 6. Kapitel behandelt Infrastruktur, Deployment und Betrieb. Es wird auf die Container-Orchestrierung, CI/CD-Pipelines mit Deployment, Monitoring-Strategien sowie die Backup- und Restore-Strategie eingegangen. Auch wird kurz die MkDocs-basierte Entwicklerdokumentation erwähnt.

Kapitel 7 widmet sich der Qualitätssicherung durch Tests und der Funktionsweise der Kennzeichenerkennung unter realen Bedingungen.

Das letzte Kapitel schließt mit einer Zusammenfassung und Evaluierung der Ergebnisse, der kritischen Reflexion des Projektverlaufs und einem Ausblick auf mögliche Erweiterungen.



## 2 Theoretische Grundlagen

### 2.1 Containerisierung (Docker)

Die Containerisierung ist eine Methode, bei welcher Anwendungen und deren Abhängigkeiten in einer leichtgewichtigen, isolierten Einheit, einem sogenannten Container, verpackt werden.

#### 2.1.1 Docker

Docker ist eine offene Plattform, welche 2013 veröffentlicht wurde und die Containerisierung von Anwendungen standardisiert. Es stellt die notwendigen Tools bereit, um Container einfach zu erstellen und mit ihnen arbeiten zu können, und abstrahiert dabei die Komplexität der zugrundeliegenden Linux-Kernel-Features (Namespaces, cgroups). Die Docker-Plattform besteht aus der Docker Engine (Daemon, REST-API und CLI), Docker Images und Docker Containern. **[docker\_overview]**



**Bild 1:** Docker Logo

Quelle: <https://www.docker.com/company/newsroom/media-resources/>

#### 2.1.2 Architektur und Funktionsweise

Docker basiert auf einer Client-Server-Architektur. Der Docker-Daemon verwaltet Container, Images, Netzwerke und Volumes (persistenter Speicher), während der Docker-Client (CLI) die Schnittstelle für Benutzerinteraktionen bereitstellt. **[docker\_overview]**

Images, also Abbilder der Anwendung und ihrer Umgebung, werden als unveränderliche Vorlagen aus sogenannten Dockerfiles erstellt und in Layern organisiert, wobei bestimmte Instruktionen wie RUN oder COPY im Dockerfile jeweils einen neuen Layer erzeugen. **[docker\_dockerfile]** Dieses Layer-System ermöglicht effizientes Caching: Wenn ein Image neu gebaut werden muss, müssen nur die Layer neu erstellt werden, an denen tatsächlich Änderungen vorgenommen wurden. **[docker\_cache]**

Container werden aus diesen Images instanziiert und stellen isolierte Prozesse und deren Umgebung dar. Im Gegensatz zu virtuellen Maschinen, die jeweils ein vollständiges Gastbetriebssystem benötigen, teilen sich Container den Host-Kernel, was sie deutlich ressourcenschonender macht. **[docker\_overview]**

### 2.1.3 Vorteile von Containerisierung und Docker

Der wohl größte Punkt, welcher für Containerisierung spricht, ist die Eliminierung des klassischen „Works on my machine“-Problems. Der Container läuft in der Entwicklung identisch zur Produktion, da das gesamte Betriebssystem, einschließlich Konfigurationen, Bibliotheken und Abhängigkeiten, im Image verpackt sind. Dies reduziert Deployment-Fehler erheblich und erleichtert das Debugging, da Entwickler exakt dieselbe Umgebung lokal reproduzieren können.

Ohne Nutzung von Containerisierung müssten die einzelnen Services entweder als Programme verpackt werden (zum Beispiel als Windows .exe-File) oder der gesamte Programmcode muss auf dem Ziel-System ausgerollt und gestartet werden, was das oben genannte „Works on my machine“-Problem mit sich bringt.

Wie zuvor erwähnt läuft jeder Service (Grafana, PostgreSQL, Python-Services etc.) in einer vollständig isolierten Umgebung. Abhängigkeiten oder Python-Pakete (etwa definiert in einem `uv.lock`-File) eines Services können nicht mit denen eines anderen kollidieren. Die Dateisystem-Isolation stellt sicher, dass jeder Container sein eigenes Root-Dateisystem hat, wodurch versehentliche Überschreibungen oder unbeabsichtigte Datenzugriffe verhindert werden. Auch sind Container auf Netzwerk-Ebene isoliert, bedeutet also, dass die Kommunikation zwischen Containern und der „Außenwelt“ nur über definierte Schnittstellen stattfinden kann.

Ein weiterer Vorteil ist die Versionskontrolle und Rollback-Fähigkeit, welche Docker-Images bieten. Diese können mit Tags versehen werden, wodurch spezifische Versionen einer Anwendung eindeutig identifizierbar sind. Bei Problemen, etwa nach einem Update, kann somit schnell zu einer vorherigen, stabilen Version zurückgerollt werden.

Zudem bieten Container eine hohe Skalierbarkeit. So können Anwendungen sehr leicht horizontal skaliert werden, indem mehrere Instanzen derselben Services gestartet werden. In Verbindung mit Orchestrierungstools wie Kubernetes können Services dynamisch basierend auf der aktuellen Last skaliert werden.

### 2.1.4 Docker Compose

Docker Compose ist ein Tool zur Definition und Verwaltung von Multi-Container-Anwendungen. Compose ermöglicht die Orchestrierung mehrerer Services durch eine deklarative YAML-Konfigurationsdatei. In dieser Datei werden alle Services, Abhängigkeiten, Netzwerke, Volumes und Umgebungsvariablen definiert. Docker Compose startet Services in der vordefinierten Reihenfolge und ermöglicht automatische Kommunikation zwischen ihnen. **[docker\_compose]** Besonders wertvoll ist Docker Compose für Entwicklungsumgebungen, da es die gesamte Infrastruktur als Code (Infrastructure as Code) definiert. Statt mehrere Services manuell zu starten, orchestriert eine einzige Datei das komplette System, was auch unterschiedliche Umgebungen wie etwa Entwicklung und Live (Produktion) durch separate Compose-Dateien ermöglicht.

### 2.1.5 Docker Hub

Docker Hub ist die zentrale, cloudbasierte Registry für Docker-Images und fungiert als öffentliches Repository, ähnlich wie GitHub für Code. Es enthält eine Vielzahl an vorgefertigten Images, darunter offizielle Images für gängige Software und Technologien wie PostgreSQL, Redis oder Python, welche von den jeweiligen Maintainern verifiziert werden. **[docker\_hub]** Beim Starten eines Stacks oder Containers lädt Docker automatisch benötigte Images von Docker Hub herunter, falls sie nicht lokal vorhanden sind. In dieser Diplomarbeit werden beispielsweise offizielle Images für PostgreSQL und Grafana verwendet, während selbsterstellte Images für die Python-Services verwendet werden, welche auf einer eigenen Docker-Registry gespeichert sind.

Für diese Diplomarbeit bietet sich Containerisierung sehr gut an, da das gesamte System mit einem einzigen Befehl auf jedem System orchestriert gestartet, gestoppt oder neu gebaut werden kann, unabhängig vom Host-Betriebssystem.

## 2.2 Microservice Architektur

Die Microservice-Architektur ist ein Designansatz, bei dem eine Anwendung als Sammlung kleiner, lose gekoppelter Dienste aufgebaut wird.

### 2.2.1 Grundkonzept und Definition

Während zum Beispiel eine monolithische Architektur alle Funktionen einer Anwendung in einem einzigen, zusammenhängenden Prozess vereint, werden bei Microservices einzelne Funktionsbereiche in eigenständige Services ausgelagert. Jeder dieser Services läuft in einem eigenen Prozess, besitzt eine klar definierte Schnittstelle und kommuniziert über leichtgewichtige Protokolle wie REST/HTTP oder Message Queues, wie RabbitMQ, mit anderen Services. **[aws\_microservices]** In diesem Projekt erfolgt die Kommunikation primär über RESTful APIs, wobei jeder Service seine Endpunkte bereitstellt.

Der Begriff „Microservice“ bezieht sich dabei weniger auf die absolute Größe des Codes, sondern vielmehr auf die fokussierte Verantwortung: Ein Service sollte genau eine Aufgabe erfüllen und diese gut (Single Responsibility Principle). Jeder Microservice ist eine eigenständige Einheit mit eigener Datenhaltung, Geschäftslogik und oft auch einem eigenen Technologie-Stack. **[oso\_microservices]**

### 2.2.2 Vorteile der Microservice-Architektur

Ein fundamentaler Vorteil von Microservices ist die Möglichkeit, Services unabhängig voneinander zu deployen. Wenn eine Änderung an einem Service vorgenommen wird, kann dieser Service aktualisiert werden, ohne dass andere Services neu gestartet oder verändert werden müssen. Dieses Prinzip profitiert stark von der Containerisierung, welche im letzten Kapitel erklärt wurde. **[aws\_microservices]**

Ein weiterer Vorteil von Microservices ist die Technologiefreiheit, welche sie bieten. Jeder Service kann in der für seine Anforderungen am besten geeigneten Programmiersprache und mit dem optimalen Framework entwickelt werden. Während die Python-Services in diesem Projekt etwa FastAPI für die API-Entwicklung nutzen, könnte ein zukünftiger Service in Go geschrieben werden, oder ein Frontend-Service könnte Next.js verwenden.

Auch wird durch die Isolation der Services die Fehlerausbreitung begrenzt. Etwa kann, wenn ein ganzer Service (wie der für Benachrichtigungen) abstürzen sollte oder nicht erreichbar ist, die Kernfunktion des Systems, also die Kennzeichenerfassung ohne Unterbrechungen weiterarbeiten. Das System kann mit degradierter Funktionalität weiterlaufen, anstatt vollständig auszufallen, was zu höherer Fehlertoleranz und Resilience führt.

### 2.2.3 Wartbarkeit und Verständlichkeit

Microservices ermöglichen es, dass Entwickler unabhängig voneinander arbeiten können. Jeder kann für einen oder mehrere Services verantwortlich sein und autonom Entscheidungen über Technologien, Entwicklungsprozesse und Deployment-Strategien treffen. Kleinere, fokussierte Codebasen sind auch einfacher zu verstehen und zu warten als ein großer Monolith. Neue Entwickler können sich schnell in einen einzelnen Service einarbeiten, ohne das gesamte System verstehen zu müssen. Auch Refactorings oder große Änderungen sind risikoärmer, da Änderungen auf einen Service beschränkt bleiben.

Die Kombination von Microservices mit Docker-Containerisierung verstärkt diese Vorteile noch: Jeder Service läuft in seiner eigenen isolierten Umgebung, Dependencies sind klar definiert, und die gesamte Infrastruktur kann als Code versioniert und reproduzierbar deployed werden (siehe Kapitel 2.1.4).

Die Implementierung einer Microservice Architektur bringt auch einige Herausforderungen und Trade-offs mit sich. Diese beziehen sich primär auf die erhöhte Komplexität, welche die Verteilung der Funktionalität unausweichlich mit sich bringt und ein gewisser Overhead, der entsteht, da jeder Service in einer separaten Instanz (Container) gestartet werden muss.

Im Fazit ist die Microservice-Architektur ein mächtiger Ansatz für komplexe Systeme, der Flexibilität, Fehlertoleranz und Skalierbarkeit bietet und in dieser Diplomarbeit vor allem in Bezug auf Fehlertoleranz, wie es oben beschrieben wurde, einen großen Vorteil mit sich bringt.

## 2.3 ALPR (Automatic License Plate Recognition)

ALPR (Automatic License Plate Recognition) bezeichnet die automatisierte Erfassung und Auswertung von Fahrzeugkennzeichen aus Bilddaten mittels Computer-Vision und Deep Learning. [platerecognizer\_alpr] ALPR-Systeme arbeiten kontinuierlich und transformieren visuelle Informationen wie Kamera-Snapshots in strukturierte, maschinell verarbeitbare Daten und ermöglichen damit die automatische Identifikation von Fahrzeugen.

### 2.3.1 Grundfunktionsweise

Der Erkennungsprozess und die Verarbeitung der Daten erfolgt typischerweise in einem mehrstufigen Prozess [senstar\_alpr]:

1. **Detection (Lokalisierung):** Das System identifiziert zunächst relevante Bildbereiche. Moderne Ansätze nutzen hierfür Object Detection-Algorithmen, um Fahrzeuge im Gesamtbild zu lokalisieren und anschließend den spezifischen Kennzeichenbereich zu extrahieren. Diese Vorselektion reduziert die zu verarbeitende Datenmenge erheblich und fokussiert nachfolgende Verarbeitungsschritte nur auf relevante Regionen.
2. **Transformation (Normalisierung):** Kennzeichen werden selten in perfekter Frontalansicht und bei besten Lichtverhältnissen erfasst. Für eine zuverlässige Erkennung müssen Parameter wie Perspektive, Kontrast, Helligkeit oder andere Bildattribute angepasst werden, um die Sichtbarkeit des Kennzeichens zu verbessern.
3. **OCR (Optical Character Recognition):** Die eigentliche Zeichenerkennung wandelt Bildinformationen in Text um. Diese nutzen Deep Learning-basierte Ansätze, die einzelne Zeichen nicht nur isoliert betrachten, sondern auch den Kontext benachbarter Zeichen berücksichtigen. Jedes erkannte Zeichen wird mit einem Konfidenzwert versehen, der die Erkennungssicherheit quantifiziert.
4. **Post-Processing (Validierung):** Die Rohergebnisse werden gegen bekannte Muster und Regeln validiert, wie etwa länderspezifische Kennzeichensyntaxen (etwa die österreichische Struktur aus Bezirk und Erkennungsnummer), diese dienen als Prüfung der Plausibilität der erkannten Nummerntafel. Auch werden typische OCR-Verwechslungen wie „0“ versus „O“ oder „1“ versus „l“ kontextbasiert korrigiert.

### 2.3.2 Technologie und Anbieter

In diesem System wird der Dienst Plate Recognizer eingesetzt, der moderne Deep Learning-Ansätze nutzt und im Vergleich zu anderen Optionen wie etwa einem Offline-OCR-Modell über klassische Texterkennung hinausgeht.

### 2.3.3 MMC-Erkennung (Make, Model, Color)

Neben der im Vergleich sehr zuverlässigen Kennzeichenerkennung identifiziert Plate Recognizer zusätzliche Fahrzeugattribute [**platerecognizer\_mmc**], ein Ansatz, der MMC-Erkennung genannt wird:

- **Make (Marke):** Identifikation des Fahrzeugherstellers basierend auf charakteristischen Designmerkmalen.
- **Model (Modell):** Feinere Unterscheidung innerhalb einer Marke, etwa zwischen verschiedenen Baureihen oder Generationen.
- **Color (Farbe):** Bestimmung der Primärfarbe unter Kompensation von Lichtverhältnissen und Unterscheidung zwischen verschiedenen Lackierungstypen.

### 2.3.4 On-Premise SDK

Plate Recognizer bietet sowohl Cloud-basierte APIs als auch ein On-Premise SDK an. Die On-Premise-Variante ermöglicht vollständige lokale Ausführung ohne externe Netzwerkhängigkeit [**platerecognizer\_snapshot**], eine Anforderung, welche Zotter Schokolade sehr wichtig war.

**Vorteile der lokalen Ausführung:** Der größte Vorteil in der Offline-Verarbeitung der Bilddaten liegt darin, dass die potenziell personenbezogene Informationen niemals das lokale Netzwerk des Unternehmens verlassen. Dies ist besonders relevant für europäische Datenschutzanforderungen wie die DSGVO (Datenschutzgrundverordnung), die strenge Regeln für die Verarbeitung solcher Daten vorsehen. Außerdem funktioniert das System unabhängig von einer Internetverbindung, was Zuverlässigkeit auch mit eingeschränkter Konnektivität garantiert.

Die ALPR bildet das Herzstück dieser Diplomarbeit, weshalb ein zuverlässiges, datenschutzkonformes und performantes System essenziell ist.



## 3 Hardware- und Technologieauswahl

### 3.1 Hardwareauswahl

Die Auswahl der Hardware ist entscheidend für die Zuverlässigkeit und Präzision der Kennzeichenerkennung und des Gesamtsystems. Diese beschränkt sich für diese Diplomarbeit auf zwei primäre Komponenten: Einerseits wird eine Kamera benötigt, welche Live-Daten von der Ein- und Ausfahrt des Parkplatzes erfasst und diese weitersenden kann. Zudem ein Server oder ähnliches, auf dem die Services der Diplomarbeit deployed werden können, und welcher für die rechenintensiven Aufgaben zuständig ist.

Das folgende Kapitel soll einen Einblick in den Entscheidungsprozess bieten, nach dem die benötigte Hardware ausgewählt wurde. Hierfür werden zunächst die Grundanforderungen an das Gesamtsystem aus der Aufgabenstellung (Abschnitt 1.3) abgeleitet, darauf aufbauend werden dann die spezifischen Anforderungen an die Hardwarekomponenten dargelegt. Danach werden die einzelnen, im Auswahlprozess evaluierten Optionen gegenübergestellt und hinsichtlich ihrer Eignung analysiert.



**Bild 2:** Luftbild des Hauptgebäudes und Parkplatz der Zotter Schokolade GmbH

Quelle: Google, Google Earth: <https://earth.google.com/>



### 3.1.1 Anforderungsanalyse an die Hardware

**Grundanforderungen** Wie oben kurz erwähnt, existieren einige Anforderungen, welche für die Funktionsfähigkeit des Systems unausweichlich sind. Zunächst galt es, die Gegebenheiten vor Ort zu betrachten, also Lage und Aufbau des Bereiches, in dem der Fahrzeugverkehr erkannt werden sollte.

Wie das Luftbild (Abb. 2) zeigt, verfügt der Parkplatz über eine kombinierte Ein- und Ausfahrt. Dies reduziert zwar den Installationsaufwand der Hardware, erhöht jedoch die Software-Komplexität, da die Bewegungsrichtung der Fahrzeuge softwareseitig erkannt werden muss. Das Zeitintervall zwischen zwei aufeinanderfolgenden Fahrzeugen wurde aufgrund der Breite der Einfahrt auf zwei bis drei Sekunden geschätzt. Das System sollte folglich in der Lage sein, 20 bis 30 Bilder pro Minute zu analysieren und die daraus gewonnenen Daten persistent zu speichern.

**Anforderungen Kamera** Da die Kamera im Außenbereich eingesetzt wird, ist eine gewisse Witterungsbeständigkeit zwingend erforderlich. Aufgrund der notwendigen Positionierung der Kamera sollte diese mindestens die Schutzklasse IP66 (Ingress Protection) erfüllen, was einem Schutz vor starken Wasserstrahlen aus allen Richtungen entspricht.

#### [baseus\_ip\_ratings]

Da das Kennzeichenerkennungssystem primär für die Erfassung von Kunden-Kennzeichen konzipiert ist, ist von einer effektiven Hauptbetriebszeit während der Öffnungszeiten der Zotter-Erlebniswelt ausgegangen worden. Diese betragen: Mo–Fr: 9–18 Uhr, Sa: 9–18:30 Uhr. Aufgrund dessen lag im Entscheidungsprozess kein großes Augenmerk auf den Nachsichtfähigkeiten des Kamerasystems.

**Anforderungen Server** Die notwendige Rechenleistung des Servers hängt sehr stark von der Komplexität und Art der Prozesse ab, welche auf diesem ausgerollt und aktiv sind. Deshalb wurde diese anfangs absichtlich nicht genau definiert. Festgelegt wurde jedoch der Einsatz eines Linux-Betriebssystems, da dieses dem internen Unternehmensstandard entspricht, einen geringen OS-Overhead aufweist und eine breite Kompatibilität bietet. Dies wurde allerdings als zweitrangig betrachtet, da das System durch die Verwendung von Containerisierung mit Docker ohnehin eine Plattformunabhängigkeit bietet. Auch wurde seitens Zotter vorausgesetzt, dass sämtliche Daten nicht nur aus Datenschutzgründen On-Premise, sondern für eine bessere Isolation des Systems lokal auf dem eingesetzten Server gespeichert werden sollen.

### 3.1.2 Evaluierung der Hardware-Optionen

**Kamera – Auswahl** In Bezug auf die Kamera wurden von Anfang an nur IP-Kameras in Betracht gezogen, also Kameras, bei denen Video-Streams und Kommunikation per Ethernet stattfinden. Dies hat zwei Hauptgründe: Daten können zunächst in den meisten Fällen direkt per API-Requests ausgelesen werden, entweder über HTTP/REST-APIs oder im Fall von Video-Streams über RTSP (Real Time Streaming Protocol). Zudem nutzen viele IP-Kameras PoE (Power over Ethernet) für ihre Spannungsversorgung, was die Aufbaukomplexität weiter minimiert, da nur ein Kabel, welches Strom und Daten überträgt, verlegt werden muss. [wiki\_ip\_camera]

Vor dem offiziellen Start der Diplomarbeit wurden hierzu schon einige Überlegungen angestellt und mögliche Produkt-Kandidaten vorgemerkt. Im Zuge von Gesprächen mit Michael Zotter wurde darauf aufmerksam gemacht, dass bereits einige IP-Kameras des Herstellers Synology für frühere Testversuche angeschafft wurden. Besonders das Modell BC500, von welchem einige Stück bereits im Besitz der Firma Zotter waren, erschien als besonders geeignet. [synology\_bc500]



**Bild 3:** Synology BC500

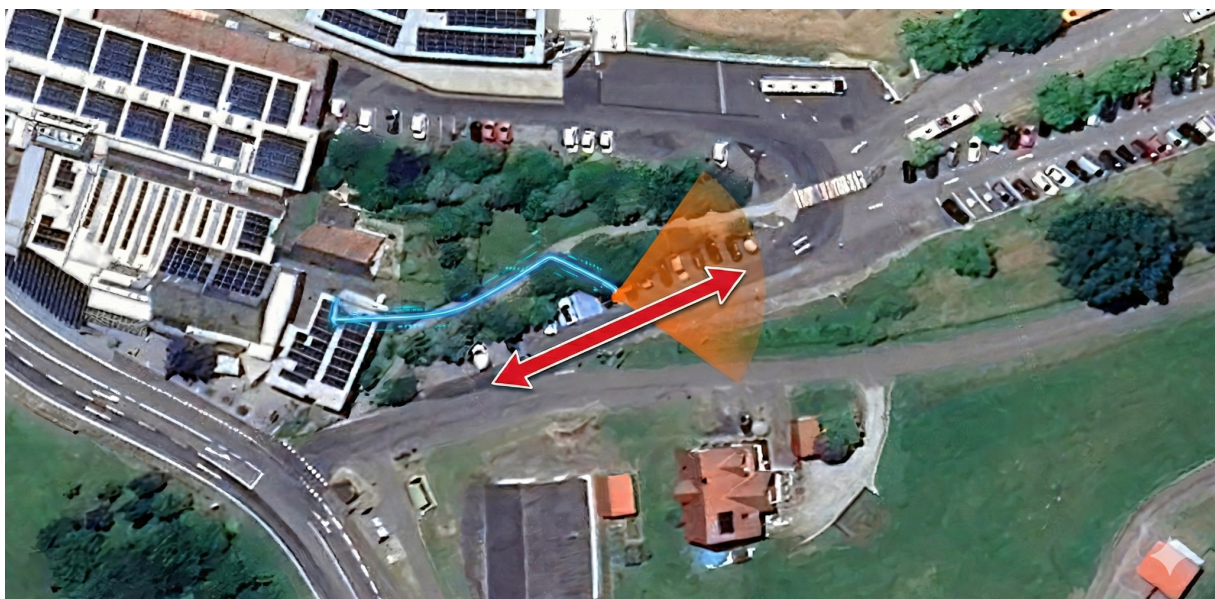
Quelle: <https://fileres.synology.com/images/ec/products/BC500.jpg>

Mit einer Witterungsbeständigkeitsbewertung von IP67, da als Außenkamera konzipiert, ist diese gut für den Außeneinsatz geeignet. Die maximale Videoauflösung von  $2880 \times 1620$  Pixel bei maximal 30 FPS erschien auch für eine konsistente Kennzeichenerkennung ausreichend. Auch verfügt die BC500 über eine automatische Nachtsichtfunktion mittels Infrarot-LEDs. Zusätzlich schien die BC500 über eine robuste API für das Auslesen von Daten und das Streamen des Feeds zu verfügen.

Aufgrund der oben angesprochenen sofortigen Verfügbarkeit und des wegfallenden Mehraufwands für diese Kamera, verbunden mit den genannten Spezifikationen, fiel die Entscheidung nach Absprache mit unserer Kontaktperson auf das Synology-Modell.

**Kamera – Montage** Das Kamerasystem wurde, wie unten erkenntlich (Abb. 4), so montiert, dass es in Richtung des Parkplatzes zeigt. Dies erwies sich als die optimale Wahl, da so der mögliche Erkennungsbereich maximiert wird und die Montage an dieser Position einfacher realisierbar war. Bei der genauen Positionierung und Ausrichtung der Kamera wurde versucht, sich so gut wie möglich am Kamera-Setup-Guide von Plate Recognizer zu orientieren. [platerecognizer\_camera\_setup]

Die erforderliche Netzwerkverkabelung wurde, wie oben im Bild ersichtlich, über den bereits vorhandenen Kabelkanal der Sicherheitsschranke geführt. Dieser verläuft bis in den Netzwerkschrank im angrenzenden Bürogebäude, wo die Anbindung an das Firmennetzwerk erfolgt.



**Bild 4:** Markup der Kamerapositionierung

Quelle: Google, Google Earth: <https://earth.google.com/>

**Server – Auswahl** Die gewählte Microservice-Architektur im Zusammenspiel mit der angewandten Containerisierung erlaubt eine hohe Flexibilität für die Art der benötigten Hardware. Des Weiteren war aus einem früheren Praktikum bei Zotter Schokolade bekannt, dass die hausinterne Infrastruktur Kapazitäten für Container-Deployment bietet.

Als zweite Option wurde das Einrichten eines kleinen Servers auf der Hardware eines Office-Mini-PCs erwogen. Dieser wäre aus reiner Implementierungs- und Funktionssicht mit der ersten Option gleichwertig, mit dem Vorteil, dass dieser Ansatz eine gewisse Unabhängigkeit zum internen Netzwerk und zur zentralen Infrastruktur ermöglicht.

An diesem Punkt wurde im Kamera-Entscheidungsprozess auf die bestehenden Synology-Produkte aufmerksam gemacht. Mit dieser Neuausrichtung kam die Idee auf, das System auf einem eigenen NAS-System (Network-attached Storage), ebenfalls von Synology zu betreiben.

Nicht nur wurden NAS-Systeme von Synology bereits im Unternehmen eingesetzt, diese sind in den meisten Bereichen gleichwertig mit den vorherigen beiden diskutierten Ansätzen und bieten in manchen Bereichen klare Vorteile [**aws\_nas**]:

1. Die Vorteile der zweiten Option bleiben bestehen, es gibt weiterhin eine gesteigerte Unabhängigkeit zur Unternehmensinfrastruktur und damit hergehend mehr Entwicklungsfreiheit.
2. Da es sich um ein NAS-System handelt, also ein Gerät, welches primär für die Speicherung von Daten konzipiert ist, stellt der benötigte Speicherplatz kein Problem mehr dar, da einfach neue Disks verbaut werden können (z.B. 2 × 2TB Disks, RAID 1).
3. Diese NAS-Systeme besitzen bereits leistungsstarke Komponenten, welche in Bezug auf Rechenleistung meist gleichwertig zu einem durchschnittlichen Mini-PC sind. Auch sind die meisten NAS-Geräte mit einem vollwertigen Betriebssystem ausgestattet, welches Methoden bereitstellt, um Container mittels Docker reibungslos laufen zu lassen.

Der größte Vorteil kommt aber mit der Kombination mit den oben beschriebenen Kameras des gleichnamigen Herstellers. Wie die Kameralinie von Synology vermuten lässt, bietet der Hersteller eigene Produkte gezielt für Überwachungsaufgaben an. Diese laufen unter einem Gesamtsystem mit der Bezeichnung „Surveillance Station“ und sind als Paket auf den hauseigenen NAS-Systemen verfügbar. [**synology\_surveillance**]

Innerhalb des Surveillance Station-Ökosystems bietet Synology mit der DVA-Serie (Deep Video Analytics) spezialisierte NVR-Geräte (Network Video Recorder) an, welche über GPU-beschleunigte KI-Funktionen verfügen. Insbesondere das Modell DVA1622 (zur Zeit des Projekts das einzige verfügbare Modell der DVA-Serie) bietet für diese Diplomarbeit in Kombination mit der Kamera BC500 viele Vorteile: **[synology\_dva]**



**Bild 5:** Synology DVA1622

Quelle: <https://www.synology.com/img/products/detail/DVA1622/heading.png>

Zunächst bietet die Surveillance Station API Zugriff auf Live-Streams, Snapshots und Ereignis-Metadaten, was die Integration in eigene Microservices im Vergleich zum direkten Auslesen aus Kameras erheblich vereinfacht. Die DVA-Serie unterstützt nativ viele Features aus dem Security- und Surveillance-Bereich. So können etwa mittels Erkennungszonen bereits auf Surveillance Station-Ebene relevante Bildbereiche wie der Einfahrtsbereich definiert werden. **[synology\_dva]** Dies ist besonders für das Erstellen von „Triggers“ relevant, also das Anlegen eines Bereiches, in dem im Falle einer Fahrzeugdurchfahrt die Kennzeichenerkennung stattfinden soll. Des Weiteren können alle Kamera-Einstellungen, Aufzeichnungen und Analysen über eine einheitliche Oberfläche auf dem NAS-Gerät verwaltet werden. **[synology\_surveillance]**

Die Kombination aus relevanten Features, umfassender API und nativer Kamera-Unterstützung machte das Synology DVA1622-NAS zur optimalen Wahl für die Realisierung des Kennzeichenerkennungssystems.

**Server – Montage** Das NAS-System ist durch seine Kompaktheit und Kapselung auf keine besonderen Anforderungen an den Montage- bzw. Betriebsort angewiesen. Aus diesem Grund wurde dieses innerhalb des Bürogebäudes installiert, in welchem auch die Kamera an das Firmennetzwerk angeschlossen ist.



## 3.2 Technik-Stack und Technologieauswahl der Kennzeichenerkennung

### 3.2.1 Technologieauswahl Kennzeichenerkennung

Für die eigentliche Kennzeichenerkennung stand die Entscheidung zwischen der Entwicklung einer eigenen Lösung auf Basis von Open-Source-OCR-Bibliotheken (wie Tesseract OCR oder EasyOCR) und der Verwendung einer kommerziellen Lösung im Raum.

Bei der Evaluation von Tesseract und EasyOCR zeigte sich, dass diese generischen OCR-Engines zwar für Texterkennungsaufgaben gut geeignet sind, bei der spezifischen Herausforderung der Kennzeichenerkennung jedoch erhebliche Nachteile aufweisen. Kennzeichen stellen besondere Anforderungen an ein Erkennungssystem, wie im Kapitel 2.3 bereits erwähnt: Sie müssen aus verschiedenen Winkeln, bei unterschiedlichen Lichtverhältnissen, mit Verschmutzungen und in Bewegung zuverlässig erkannt werden. Zudem variieren Kennzeichenformate international stark in Schriftart, Layout und Zeichenabständen.

Als weitere Option wurde die Verwendung der Synology-nativen Lösung zur Kennzeichenerkennung erwogen. Dies erwies sich aber schnell als nicht umsetzbar, da die Erkennung laut Hersteller auf dem Modell DVA1622 nur für stationäre Fahrzeuge ausgelegt ist.

#### [synology\_lpr]

Die Entwicklung eines eigenen trainierten Modells auf Basis dieser Bibliotheken hätte einen erheblichen Aufwand für das Sammeln und Annotieren von Trainingsdaten bedeutet, wobei die resultierende Genauigkeit dennoch fraglich geblieben wäre. In diese Richtung wurden im Zuge der Diplomarbeit-Entwicklung Prototypen erstellt, diese blieben aber in ihrer Qualität unzureichend oder waren in ihrem Realisierungsaufwand unrealistisch.

Aus diesem Grund fiel die Entscheidung auf Plate Recognizer, eine spezialisierte kommerzielle Lösung für Kennzeichenerkennung. Plate Recognizer bietet hochoptimierte Machine-Learning-Modelle, welche bereits auf Kennzeichenbildern aus über 90 Ländern trainiert wurden. [platerrecognizer\_countries] Dies gewährleistet eine deutlich höhere Erkennungsrate auch unter erschwerten Bedingungen [platerrecognizer\_results]. Wie im Kapitel 2.3.3 erwähnt bietet diese Lösung, zusätzlich zur reinen Texterkennung, erweiterte Funktionen wie Fahrzeugtyp- und Farberkennung, welche für zukünftige Erweiterungen des Systems nützlich sein können.

Ein weiterer entscheidender Vorteil ist die Verfügbarkeit als Docker-Container (Plate Recognizer SDK), der vollständig lokal auf dem DVA1622 NAS betrieben werden kann. Im Gegensatz zu Cloud-basierten API-Lösungen bleiben damit alle Bilddaten im lokalen Netzwerk, was sowohl Datenschutzanforderungen erfüllt als auch Latenzzeiten minimiert. [platerrecognizer\_sr

### 3.2.2 Argumentation der Software-Technologien Entscheidungen

Als Web-Framework wurde FastAPI auf Python gewählt **[fastapi]**, welches moderne asynchrone Request-Verarbeitung unterstützt und automatische API-Dokumentation via OpenAPI/Swagger generiert. Im Vergleich zu Flask, dem vermutlich bekanntesten Python-Framework, bietet FastAPI native Type-Hints-Unterstützung und automatische Validierung von Request-/Response-Daten durch Pydantic. **[jetbrains\_frameworks]** Gegenüber dem schwergewichtigeren Django bringt FastAPI deutlich weniger Overhead mit und ist damit besser für kleine Microservices geeignet, bei denen schlanke, spezialisierte Services bevorzugt werden.

#### **[jetbrains\_frameworks]**

Für die Datenhaltung wurde PostgreSQL als relationale Datenbank gewählt. **[postgresql]** Während NoSQL-Datenbanken wie MongoDB für hochskalierbare, dokumentenbasierte Anwendungen Vorteile bieten, sprechen im vorliegenden Fall die strukturierten Datenbeziehungen (Erkennungen, Benutzer etc.) klar für eine relationale Datenbank.

Um Datenbank-Schema Änderungen versioniert und reproduzierbar zu gestalten, wurde Alembic als Migrations-Framework gewählt **[alembic]**. Im Gegensatz zu manuellen SQL-Skripten ermöglicht dies rückverfolgbare Datenbankänderungen, die automatisiert über alle Umgebungen ausgerollt werden können.

Das Gesamtsystem basiert auf Docker und einer Microservice-Architektur. Wie zuvor detailliert erklärt, läuft jeder Dienst (Data Collection, Notification, Grafana) in seinem eigenen Container mit klar definierten Schnittstellen. Dies verhindert wie in den Theoretischen Grundlagen erklärt nicht nur Abhängigkeitskonflikte zwischen verschiedenen Services, sondern ermöglicht auch die unabhängige Skalierung einzelner Komponenten. Die Containerisierung gewährleistet zudem Plattformunabhängigkeit, da das System problemlos von der Synology-Plattform auf andere migriert werden kann, ohne dass Anpassungen am Code notwendig sind.

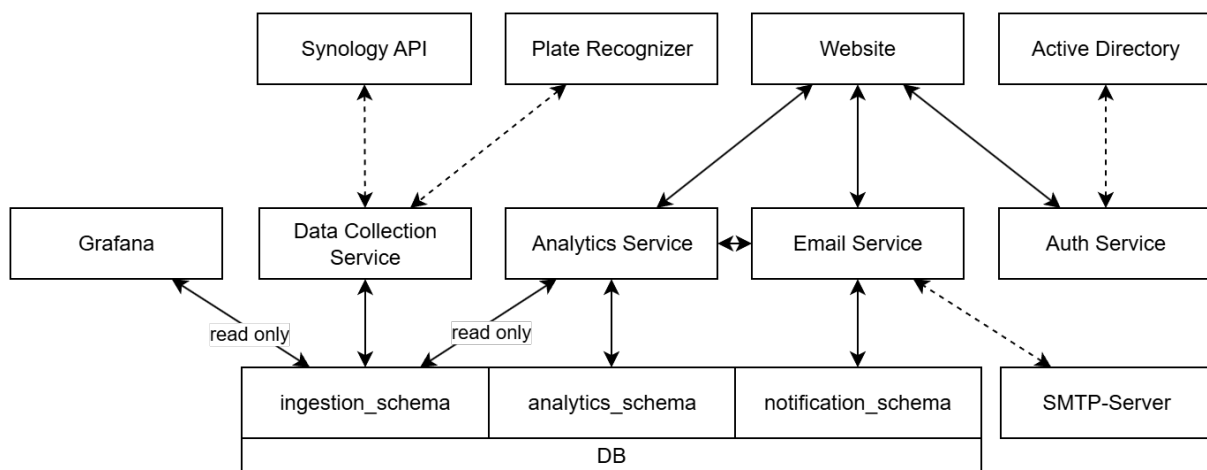
## 4 Systementwurf und Architektur

### 4.1 Gesamtarchitektur

Das System wurde als hybride Microservice-Architektur konzipiert, in der jeder funktionale Bereich als eigenständiger, containerisierter Service realisiert wird.

Für den Entwurf der Systemarchitektur wurde die Design-Methodik der „The 12-Factor App“ [twelve\_factor] herangezogen. Diese definiert eine Reihe von Best-Practices für die Entwicklung moderner Applikationen. Im Rahmen dieser Arbeit wurden insbesondere die strikte Trennung von Konfiguration und Code durch Umgebungsvariablen (Faktor III), die Behandlung der Datenbank als angebundener Backing-Service (Faktor IV) sowie die Sicherstellung der Parität zwischen Entwicklungs- und Betriebsumgebung durch Containerisierung und CI/CD (Faktor X) umgesetzt. Auf die genaue Umsetzung der Faktoren wird in den betreffenden Kapiteln genauer eingegangen.

Hybrid bedeutet in diesem Kontext primär, dass die einzelnen Services nicht ganzheitlich isoliert und entkoppelt sind, da nicht jeder dieser über eine eigene Datenhaltung verfügt. Stattdessen nutzen alle Komponenten einen gemeinsamen PostgreSQL Service, welcher auf Schema-Ebene isoliert ist. Auf dieses Konzept wird im Kapitel 4.2.1 näher eingegangen. Die Gesamtarchitektur wurde so entworfen, dass diese die in der Aufgabenstellung definierten Anforderungen abbildet. Im Zuge der Entwicklung wurden nicht alle der im folgenden Kapitel erwähnten Komponenten implementiert, die bereits konzipierten Services werden dennoch in diesem Kapitel aus Gründen der Vollständigkeit erwähnt.



**Bild 6:** Architektur Diagramm

Wie in der Abbildung (Abb. 6) erkenntlich besteht diese Diplomarbeit aus Komponenten, welche sich in drei primäre Kategorien einteilen lassen: Backend-, Frontend- und Infrastrukturservices.



Darüber hinaus bestehen Anbindungen an externe Services (wie etwa die Plate Recognizer SDK). Sofern diese implementiert wurden, werden sie im Kapitel Implementierung (Abschnitt 5) beim jeweiligen Dienst im Detail erläutert. Die Architektur folgt bewusst keinem API-Gateway Ansatz, wie es oft bei Microservice-Ansätzen typisch ist, da die Services primär intern kommunizieren und nur wenige Endpunkte nach außen hin offen zugänglich sind.

#### 4.1.1 Backend-Services

Das funktionale Backend des Systems bilden vier Python-Services basierend auf dem FastAPI-Framework. Der Data Collection Service nimmt als Herzstück des Systems Webhook-Events der Synology Surveillance Station entgegen, sobald eine Fahrzeugbewegung im definierten Erkennungsbereich erkannt wurde. Dieser beantragt daraufhin einen Kamera-Snapshot, leitet diesen an die Plate Recognizer SDK weiter und speichert die verarbeiteten, anonymisierten Ergebnisse in der Datenbank.

Der Notification Service hat Möglichkeiten, um Benutzer-Einstellungen bezüglich E-Mail-Benachrichtigungen zu verwalten und den Versand dieser via SMTP zu steuern. Dieser Service kann etwa benutzt werden, um Tagesstatistiken oder Ähnliches mit Hilfe des Analytics Service bereitzustellen.

Der eben erwähnte Analytics Service (Auswertungsservice) wurde konzipiert, um analytische Endpunkte bereitzustellen, über welche Auswertungen und Vorhersagen aus gesammelten Erkennungsdaten abgerufen werden können.

Der Auth Service soll laut Konzeption Benutzer der Web-Oberfläche gegen das unternehmensinterne Active Directory authentifizieren, um eine nahtlose Integration in die bestehende Unternehmensinfrastruktur zu ermöglichen.

#### 4.1.2 Frontend

Als Benutzeroberfläche sollte primär der Web Service, eine mit Next.js [nextjs] realisierte Web-Applikation dienen. Diese wurde als zentrale Oberfläche für Konfiguration und Verwaltung von Benachrichtigungen konzipiert und kommuniziert ausschließlich über REST-APIs mit den Backend-Services. Als Übergangslösung und zweite Visualisierungsplattform dient Grafana, in Form eines Dashboards für diverse Statistiken.

### 4.1.3 Infrastruktur-Services

Die Infrastruktur-Ebene umfasst mehrere Stützkomponenten, die den Betrieb des Gesamtsystems ermöglichen. PostgreSQL fungiert als zentrale relationale Datenbank für die persistente Datenhaltung aller Services. Der Plate Recognizer SDK-Container stellt das lokal betriebene ALPR-Modell als REST-API bereit, sodass die Bildverarbeitung vollständig im lokalen Netzwerk stattfindet wie zuvor im Kapitel 2.3.4 bereits ausführlich erklärt wurde. Ein DB-Prestart-Container übernimmt die automatische Initialisierung der Datenbank beim Systemstart. Ergänzend sorgt ein Backup-Container für regelmäßige, automatisierte Datenbanksicherungen. Die Details zu diesen Infrastruktur-Komponenten werden im Kapitel 6 näher erläutert.

### 4.1.4 Kommunikationswege

Die Kommunikation zwischen den Services erfolgt ausschließlich über RESTful HTTP-APIs. Der primäre Kommunikationsweg im System, welcher für die Datenerfassung erfolgt, lässt sich wie folgt zusammenfassen:

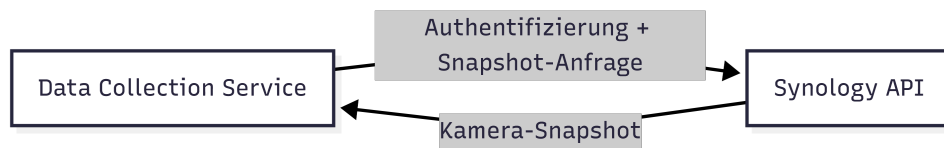
#### 1. Externer Trigger durch Erkennung eines Fahrzeugs → Data Collection Service



**Bild 7:** Trigger für den Data Collection Service

Die Synology Surveillance Station ruft bei Erkennung einer Fahrzeugbewegung einen REST-Webhook des Data Collection Service auf.

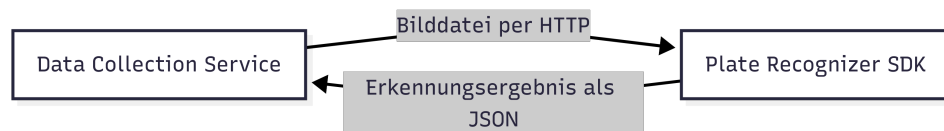
## 2. Data Collection Service → Synology API



**Bild 8:** Data Collection Service zu Synology API

Nach Empfang des Webhooks authentifiziert sich der Service bei der Synology-API, um einen aktuellen Kamera-Snapshot abzurufen.

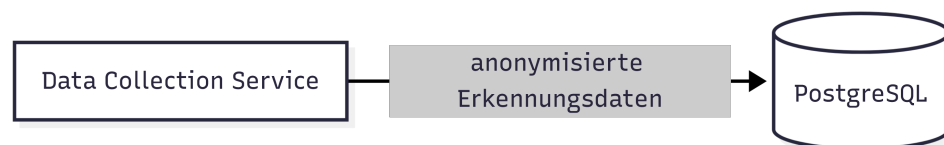
## 3. Data Collection Service → Plate Recognizer SDK



**Bild 9:** Data Collection Service zu Plate Recognizer SDK

Der abgerufene Snapshot wird als Bilddatei per HTTP an den lokal betriebenen Plate Recognizer Container gesendet, der die Kennzeichenerkennung durchführt und das Ergebnis als JSON zurückgibt.

## 4. Data Collection Service → PostgreSQL

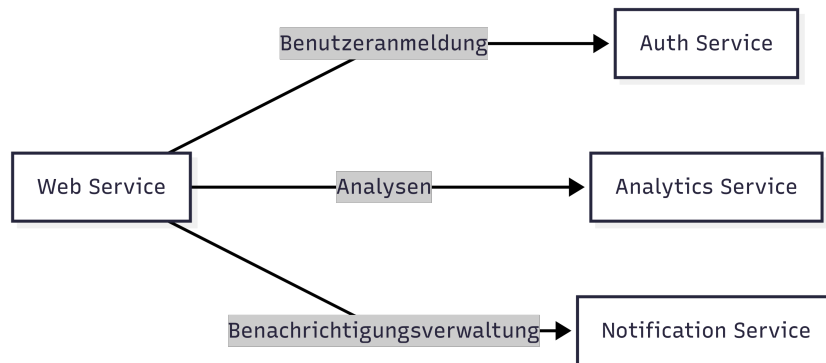


**Bild 10:** Data Collection Service zu DB

Die verarbeiteten und anonymisierten Erkennungsergebnisse werden direkt in die Datenbank geschrieben.

Weitere konzipierte, teils implementierte Verbindungen:

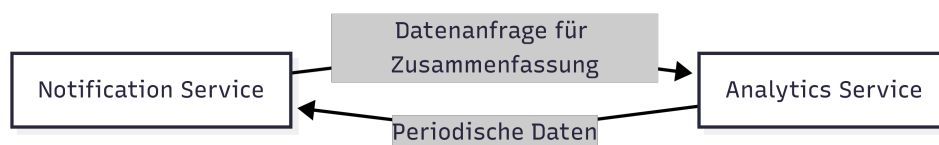
## 5. Web Service → Backend-Services



**Bild 11:** Web Service Interaktionen

Die Web-Oberfläche kommuniziert über REST-Calls mit dem Auth Service (Benutzeranmeldung), dem Analytics Service für Analysen und dem Notification Service (Benachrichtigungsverwaltung).

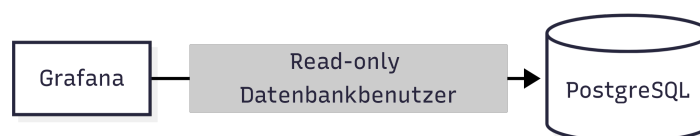
## 6. Notification Service → Analytics Service



**Bild 12:** Notification Service zu Analytics Service

Für den Versand von periodischen Zusammenfassungen ruft der Notification Service Daten vom Analytics Service ab.

## 7. Grafana → PostgreSQL



**Bild 13:** Grafana zu DB

Grafana greift über einen Read-only Datenbankbenutzer lesend auf das Data-Collection-Schema der Datenbank zu, um das Dashboard mit aktuellen Daten zu versorgen.

## 4.2 Datenbankdesign

### 4.2.1 Architekturansatz

Anstatt für jeden Microservice eine eigene PostgreSQL-Instanz zu betreiben, wurde eine Single-Database-Strategie auf Basis einer PostgreSQL-Instanz mit Schema-basierter Isolation gewählt [**postgresql\_schemas**]. Innerhalb der gemeinsam genutzten PostgreSQL-Datenbank existieren also drei, logisch getrennte Schemas:

1. Das Data-Collection-Schema (Ingestion) enthält die Kerndaten der Fahrzeugerkennungen.
2. Das Analytics-Schema (Analytics) ist für aggregierte oder vorberechnete Analysedaten konzipiert.
3. Das Notification-Schema (Notification) speichert Nutzerpräferenzen für den Benachrichtigungsdienst.

Dieser Ansatz, eine einzelne Datenbankinstanz durch Schemas logisch zu unterteilen, anstatt mehrere Instanzen zu betreiben, bietet im Kontext dieser Diplomarbeit mehrere Vorteile: Zunächst reduziert er den operativen Aufwand, da nur eine Datenbankinstanz verwaltet, gesichert und überwacht werden muss, wie es etwa bei einer klassischen Microservice-Architektur der Fall wäre. Dennoch bleibt die logische Trennung der Daten gewährleistet, da jeder Service nur auf sein zugewiesenes Schema zugreifen kann. Des Weiteren vereinfacht er Cross-Schema-Lesezugriffe, etwa kann der Analytics Service oder Grafana Abfragen direkt auf die Ingestion-Daten zugreifen, ohne dass dafür eine Datenreplikation oder zusätzliche API-Aufrufe zwischen Services nötig sind. Die Zugriffskontrolle wird dabei über dedizierte PostgreSQL-Benutzer mit minimalen Berechtigungen erzwungen, was im Abschnitt zum Sicherheitskonzept näher beleuchtet wird.

### 4.2.2 Entitäten und ER-Modell

Das Datenbankschema des Systems umfasst zwei zentrale Entitäten, welche sich in getrennten Schemas befinden.

Die Tabelle `vehicle_observations` im Data-Collection-Schema ist die zentrale Entität des gesamten Systems. Jeder Datensatz repräsentiert eine einzelne erfasste Fahrzeugerkennung und bildet die Grundlage für sämtliche Analysen und Visualisierungen.

Die Tabelle speichert neben dem Erkennungszeitpunkt und einem anonymisierten Hash des Kennzeichens auch den Konfidenzwert der Erkennung, das Herkunftsland, gegebenenfalls den Bezirk (für österreichische und slowenische Kennzeichen), Fahrzeugkategorie, Marke, Modell, Farbe sowie die Fahrtrichtung. Letztere wird durch die Orientierung des Fahrzeugs (Vorder- oder Rückseite sichtbar) ermittelt und ermöglicht damit die Unterscheidung zwischen Ein- und Ausfahrten.

Die Tabelle `user_preferences` im Notification-Schema verwaltet die Benachrichtigungseinstellungen der Systembenutzer. Sie speichert pro Benutzer den Namen und E-Mail Adresse sowie zwei Flags, welche festlegen, ob der Benutzer Echtzeit-Alerts oder periodische Zusammenfassungen per E-Mail erhalten möchte.

Die folgende Darstellung (Abb. 14) zeigt die Attribute der beiden Entitäten als ER-Diagramm:

| VEHICLE_OBSERVATIONS |              |    |
|----------------------|--------------|----|
| int                  | id           | PK |
| datetime             | timestamp    |    |
| binary               | plate_hash   |    |
| int                  | plate_score  |    |
| string               | country_code |    |
| string               | municipality |    |
| string               | vehicle_type |    |
| string               | make         |    |
| string               | model        |    |
| string               | color        |    |
| enum                 | orientation  |    |

| USER_PREFERENCES |                 |    |
|------------------|-----------------|----|
| int              | id              | PK |
| string           | name            | UK |
| string           | email           | UK |
| bool             | receive_alerts  |    |
| bool             | receive_updates |    |
| datetime         | created_at      |    |
| datetime         | updated_at      |    |

**Bild 14:** Datenbank Tabellen

### 4.2.3 Argumentation zur Denormalisierung

Das Datenbankschema der `vehicle_observations` Tabelle ist bewusst nicht vollständig normalisiert. Statt etwa `country_code`, `vehicle_type`, `make`, `model` und `color` in eigene Lookup-Tabellen mit Foreign-Key-Relationen auszulagern, werden diese direkt als Strings in der Observationstabelle gespeichert.

Diese Entscheidung beruht auf mehreren Gründen:

**1. Datenherkunft aus externer API** Die Daten werden direkt aus der JSON-Antwort der Plate-Recognizer-API übernommen. Die API gibt eine flache Struktur zurück, in welcher das Erkennungsergebnis alle Attribute bereits als einfache Zeichenketten oder Werte enthält. Das Erstellen und die Wartung eines normalisierten Schemas mit Lookup-Tabellen für dynamische, von einer externen API stammende Werte, würde unnötige Komplexität hinzufügen und einen laufenden Wartungsaufwand bedeuten, da neue Werte (z.B. neue Fahrzeugmodelle) oder etwa Benennungsanpassungen kontinuierlich eingepflegt werden müssten.

**2. Lesedominanz** Die Daten werden primär lesend konsumiert, sowohl von Grafana für Dashboard-Visualisierungen als auch vom konzipierten Analytics-Service. Bei leselastigen Workloads sind Verknüpfungen über mehrere Tabellen teurer als direkte Abfragen auf einer einzelnen, denormalisierten Tabelle.

**3. Einfachheit der Abfragen** Die Grafana-Dashboards verwenden direkte SQL-Abfragen. Eine einfache, flache Tabellenstruktur macht diese Abfragen nicht nur lesbarer, sondern eliminiert auch die Fehleranfälligkeit, die durch komplexe JOIN-Befehle entstehen könnte.

**4. Unveränderlichkeit der Daten** Fahrzeugerkennungen sind unveränderliche Ereignisse, im Gegensatz zu beispielsweise einem Produktkatalog, wo sich der Name eines Herstellers ändern kann, sind gespeicherte Erkennungen statisch. Die Hauptvorteile einer Normalisierung, etwa die konsistente Aktualisierung gemeinsamer Daten, sind daher nicht wirklich relevant.



**5. Speicherplatz als unkritische Ressource** Die durch die Denormalisierung entstehende Datenredundanz, etwa wiederholt gespeicherte Herstellernamen, führt zu einem geringfügig höheren Speicherbedarf. Dieser ist im vorliegenden Anwendungsfall jedoch vernachlässigbar, da das System auf einer NAS-Plattform mit erweiterbarem Speicher betrieben wird und die Datensatzgröße pro Erkennung minimal ist. Zum Stand der Dokumentation dieser Arbeit beträgt ein volles Backup der gesamten Datenbank (Über 60.000 Erkennungen) ca. 3.4 MB. Um diesen Punkt zu verdeutlichen, wurde die theoretische Dauer bis zur vollständigen Kapazitätsauslastung des verbauten NAS-Systems ermittelt.

Formel zur Berechnung der Speicherauffüllungsdauer anhand der Größe eines Full-Backups:

$$T = \frac{S_{\max} \cdot D_t}{G_t} \quad (1)$$

wobei  $T$  Tagen der Dauer bis zum vollständigen Befüllen eines Speicherplatzes  $S_{\max}$  in Bytes entspricht,  $G_t$  die Größe eines Full-Backups in Bytes und  $D_t$  die vergangenen Tage seit Beginn der Kennzeichenerfassungen zum Zeitpunkt  $t$  darstellen.

Als verfügbarer Speicherplatz  $S_{\max}$  wurde 1TB ( $10^{12}$  Bytes) angenommen, da dies dem halben, derzeit im NAS-System verbauten Speicher entspricht.  $G_t$  betrug (Stand 12. Februar 2026) wie bereits oben erwähnt rund 3,4MB ( $3,4 \cdot 10^6$  Bytes) und zu diesem Stand sind 188 Tage ( $D_t$ ) seit Beginn der Aufzeichnungen vergangen.

Es gilt dementsprechend:

$$T = \frac{10^{12} \cdot 188}{3,4 \cdot 10^6} \approx 5,53 \cdot 10^7 \text{ Tage} / 365,25 \approx 151387 \text{ Jahre} \quad (2)$$

Das 1TB-NAS würde daraus folgend bei gleichbleibendem Wachstum also erst in rund 151 Tausend Jahren voll sein. Die Speicherkapazität ist damit für den praktischen Betrieb als unbegrenzt zu betrachten, selbst wenn der verfügbare Speicher als erheblich kleiner angenommen wird.<sup>1</sup>

Die Herleitung dieser Formel ist dem Anhang zu entnehmen.

---

<sup>1</sup>Es ist anzumerken, dass die Größe eines Full-Backups zwar in einem linearen Zusammenhang mit der tatsächlichen Datenbankgröße steht, diese aber zum Beispiel durch Komprimierung nicht vollständig widerspiegelt [`so_backup_size`; `sqlskills_backup`]. Auch nehmen die Erkennungs-Einträge nicht den gesamten Platz des Backups ein, da andere Daten wie etwa die anderer Services ebenfalls in diesem enthalten sind. Das oben genannte Ergebnis ist aus diesen Gründen nur als Annäherung zu betrachten, wobei die Kernaussage ohne Zweifel bestehen bleibt.

## 4.3 Sicherheitskonzept

Die Sicherheitsarchitektur des Systems ist in mehrere Schichten gegliedert, die jeweils unterschiedliche Aspekte des Schutzes adressieren.

### 4.3.1 Authentifizierung und Endpunkt-Sicherheit

Die Absicherung der API-Endpunkte erfolgt je nach Service und Anforderung durch unterschiedliche Mechanismen:

#### **Data Collection Service (HTTP Basic Authentication)**

Der Webhook-Endpunkt `/api/vehicle_detected`, über den die Synology Surveillance Station Fahrzeugerkennungen meldet, ist durch HTTP Basic Authentication geschützt. Die Zugangsdaten werden dabei gegen die Synology-Credentials des Systems validiert. HTTP Basic Authentication ist dabei die einzige von der Surveillance Station unterstützte Authentifizierungsmethode für Webhook-Aufrufe, weshalb hier keine Alternative in Frage kam. Da der Endpunkt ausschließlich für die maschinenbasierte Kommunikation innerhalb des lokalen Netzwerks vorgesehen ist und nicht von außen erreichbar ist, ist dieses Sicherheitsniveau für diesen Anwendungsfall ausreichend.

#### **Notification Service (API-Key Authentication)**

Der Notification Service erwartet einen statischen API-Key im Authorization-Header jeder Anfrage. Die Entscheidung gegen eine komplexere Authentifizierung, etwa mittels JWT (JSON Web Tokens) oder Session-basierter Tokens fiel aus mehreren Gründen: Erstens findet die Kommunikation mit dem Notification Service ausschließlich intern zwischen Services statt, nicht direkt durch Endbenutzer, zweitens würde eine JWT-basierte Lösung eine Schlüsselverwaltung (Signing Keys, Token-Rotation, Expiration-Handling) erfordern, die für einen reinen Service-zu-Service-Call unnötige Komplexität hinzufügt. Die Notwendigkeit eines statischen API-Key, der über Umgebungsvariablen konfiguriert wird, kommt daher, dass der Port dieses Services zu Testzwecken freigeschaltet wurde. Für diesen Zweck bietet ein Token ein angemessenes Sicherheitsniveau bei geringer Komplexität.

### **Auth Service / Web Service (Active Directory Integration)**

Für die Authentifizierung von Benutzern der Web-Oberfläche wurde die Integration mit dem bestehenden Active Directory (AD) der Firma Zotter konzipiert. Dieser sollte mittels LDAP (Lightweight Directory Access Protocol)-Anbindung Nutzerdaten aus dem firmeninternen Microsoft-AD Server beziehen und diese für den Login-Prozess nutzen. Dies bietet den Vorteil, dass keine separate Benutzerverwaltung implementiert und gewartet werden muss und die bestehende Unternehmensinfrastruktur zur Zugangskontrolle genutzt werden kann.

### **4.3.2 Datenbankzugriffskontrolle**

Ein zentrales Element der Sicherheitsarchitektur ist die konsequente Anwendung des Principle of Least Privilege, kurz PoLP auf Datenbankebene [[wiki\\_polp](#)]. Jeder Service operiert mit einem eigenen, dedizierten PostgreSQL-Benutzer, dessen Rechte auf das absolut notwendige Minimum beschränkt sind.

Durch diese Segmentierung wird sichergestellt, dass im Falle einer Infiltration eines einzelnen Services der Zugriff auf Daten anderer Bereiche verhindert wird. Der Notification Service kann beispielsweise keine Fahrzeugerkennungsdaten auslesen, und ein Angreifer mit Zugriff auf den Analytics Service könnte keine Daten verändern, da er nur Leserechte auf das Ingestion-Schema besitzt.

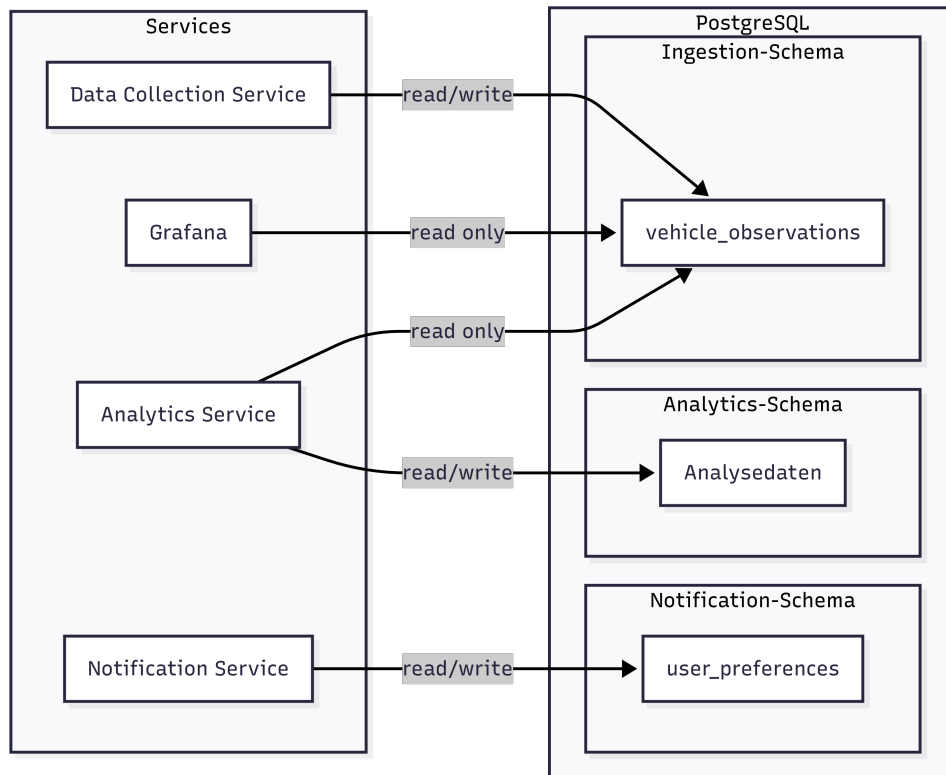


Bild 15: DB Benutzer

Wie die Darstellung (Abb. 15) zeigt, hat der Data-Collection-User Vollzugriff auf das Ingestion-Schema, der Analytics-User nur Lesezugriff darauf (aber Vollzugriff auf das eigene Analytics-Schema), und der Notification-User ist vollständig auf das Notification-Schema beschränkt. Grafana greift über den analytics\_user ebenfalls ausschließlich lesend auf die Ingestion-Daten zu.

#### 4.3.3 Datenschutz und Umgang mit Kennzeichen-Daten

Der Umgang mit Kennzeichen-Daten ist aus DSGVO-Perspektive besonders sensibel, da Kennzeichen als personenbezogene Daten klassifiziert werden [**dsgvo\_personenbezogen**]. Das System implementiert deswegen eine mehrstufige Anonymisierungsstrategie:

**1. Irreversibles Hashing** Unmittelbar nach der Erkennung wird das Kennzeichen mittels SHA-256 gehasht. Dieser Hash ist ein kryptographisch sicherer, irreversibler Einwegprozess. Heißt, aus dem gespeicherten Hash kann das Originalkennzeichen nicht mehr rekonstruiert werden, zwei erkannte Kennzeichen können dennoch auf ihre Gleichheit analysiert werden. In der Datenbank wird ausschließlich der 32-Byte-Hash (plate\_hash) gespeichert, nicht das Klartext-Kennzeichen.

**2. Keine Speicherung von Bilddaten** Die Kamera-Snapshots, aus welchen die Kennzeichen extrahiert werden, werden nicht persistent gespeichert. Die Bilddaten existieren lediglich temporär im Arbeitsspeicher, während sie zur Erkennung verarbeitet werden.

**3. Verarbeitung im lokalen Netzwerk** Wie im Kapitel zur ALPR-Technologie erläutert, läuft der Plate Recognizer als lokaler Container. Sämtliche Bilddaten und Erkennungsergebnisse verlassen zu keinem Zeitpunkt das lokale Netzwerk der Firma Zotter.

#### 4.3.4 Datenlebenszyklus (Data Lifecycle)

Der Lebenszyklus der Fahrzeugerkennung beschreibt den logischen Weg der Daten von der Erfassung bis zur langfristigen Sicherung. Das Konzept ist so gestaltet, dass personenbezogene Daten so früh wie möglich eliminiert werden und nur anonymisierte, für die Analyse relevante Informationen persistent gespeichert werden.



**Bild 16:** Detection Flow

Eine Fahrzeugbewegung im Erkennungsbereich löst ein Event aus (Erfassung), woraufhin ein Kamera-Snapshot angefordert und an die ALPR-Komponente gesendet wird (Extraktion), hierbei werden die Bilddaten nicht persistent gespeichert. Die Erkennungsergebnisse werden anschließend, falls möglich, um regionale Herkunft auf Bezirksebene angereichert. Darauf folgt die Anonymisierung, wobei das erkannte Kennzeichen mittels SHA-256 irreversibel gehasht wird. Ab diesem Punkt existiert das originale Kennzeichen in keinem Teil des Systems mehr. Der anonymisierte Datensatz wird in der PostgreSQL-Datenbank gespeichert und regelmäßig automatisiert gesichert (Details im Kapitel 6).

Durch diesen Lebenszyklus ist sichergestellt, dass personenbezogene Daten zu keinem Zeitpunkt persistent gespeichert werden und die gespeicherten, anonymisierten Daten eine Rückverfolgung auf individuelle Fahrzeuge nicht ermöglichen. Die konkrete technische Umsetzung der einzelnen Phasen wird im Kapitel 5 genauer beschrieben.

## 5 Implementierung

Dieses Kapitel dokumentiert die technische Umsetzung der in der Architektur beschriebenen Komponenten. Zunächst wird auf die Konfiguration der Synology Surveillance Station eingegangen, welche für einen korrekten Betrieb des Systems notwendig ist. Für jeden Service werden die zentralen Abläufe, Designentscheidungen und relevante Code-Auszüge dargestellt, wobei der Fokus dieses Kapitels auf der Kernkomponente, dem Data Collection Service liegt.

### 5.1 Konfiguration Synology Surveillance Station

Auf der Surveillance-Station-Plattform können alle Bereiche des Synology Überwachungssystems konfiguriert werden. Für diese Diplomarbeit sind jene Bereiche relevant, welche sich mit der Konfiguration der IP-Kameras, der Fahrzeugerkennung und der Aktionen (Trigger) befassen.

#### 5.1.1 IP Camera Konfiguration

Mit dem IP Camera-Tool können Synology-Kameras im Netzwerk automatisch erkannt und konfiguriert werden. In den Einstellungen der hinzugefügten IP-Kamera wurden Optionen wie die Bildrate oder Auflösung der Kamera maximiert, um die bestmöglichen Grundvoraussetzungen für eine erfolgreiche Erkennung zu bieten.

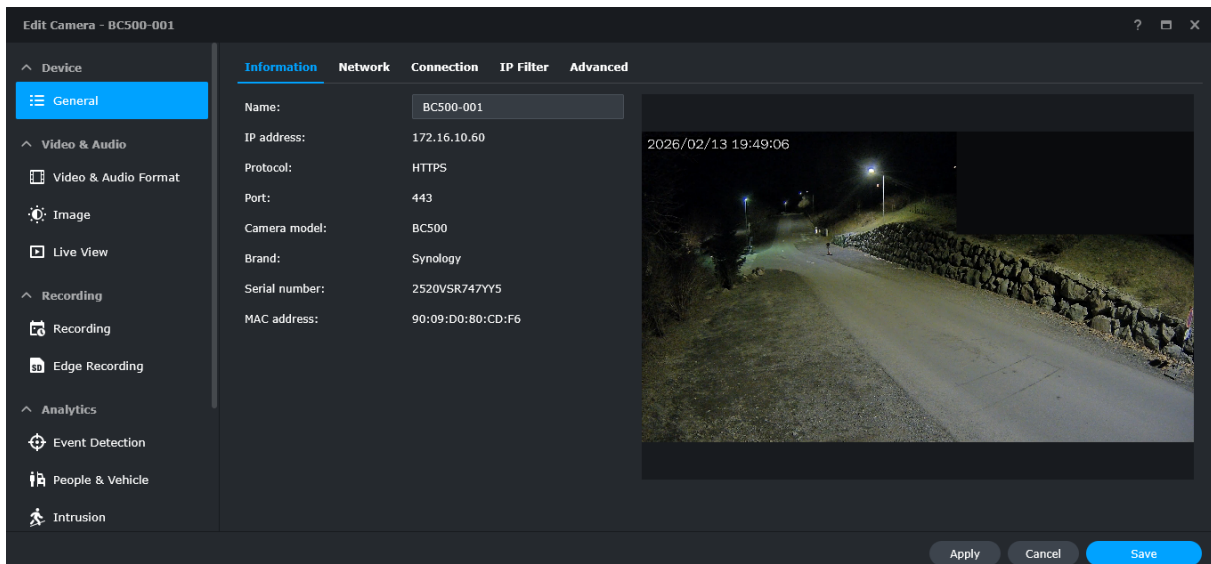
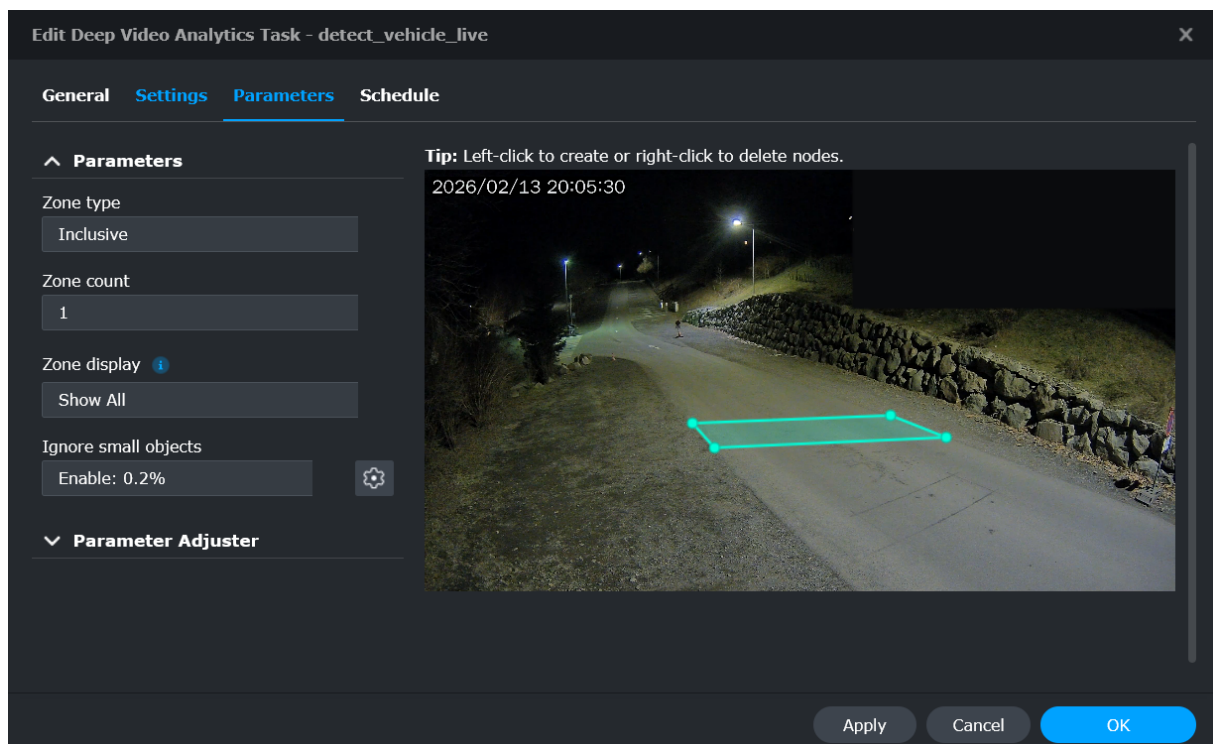


Bild 17: IP Camera Tool<sup>2</sup>

### 5.1.2 Deep Video Analytics Konfiguration

Da wie bereits im Kapitel 3 erwähnt die Synology DVA-Serie als NAS-System gewählt wurde, standen für die Umsetzung dieses Projekts die erweiterten KI-Funktionen dieser Reihe zur Verfügung, diese werden von Synology als Deep Video Analytics (DVA) bezeichnet. Eine dieser Funktionalitäten ist eine präzise und schnelle Durchfahrtskontrolle mittels Erkennungsbereichen. Diese wurde genutzt, um den unten in der Abbildung erkenntlichen Bereich zu markieren, in welchem ein Trigger ausgelöst wird, sobald ein Fahrzeug diesen durchfährt. Hierbei ist wichtig zu erwähnen, dass dieser DVA-Task so konfiguriert wurde, dass Fußgänger und stehende Fahrzeuge nicht erkannt werden. Diese Einschränkungen sind wichtig, um das ALPR-System zu entlasten und Mehrfacherkennungen durch im Bereich befindliche, stationäre Fahrzeuge zu eliminieren.

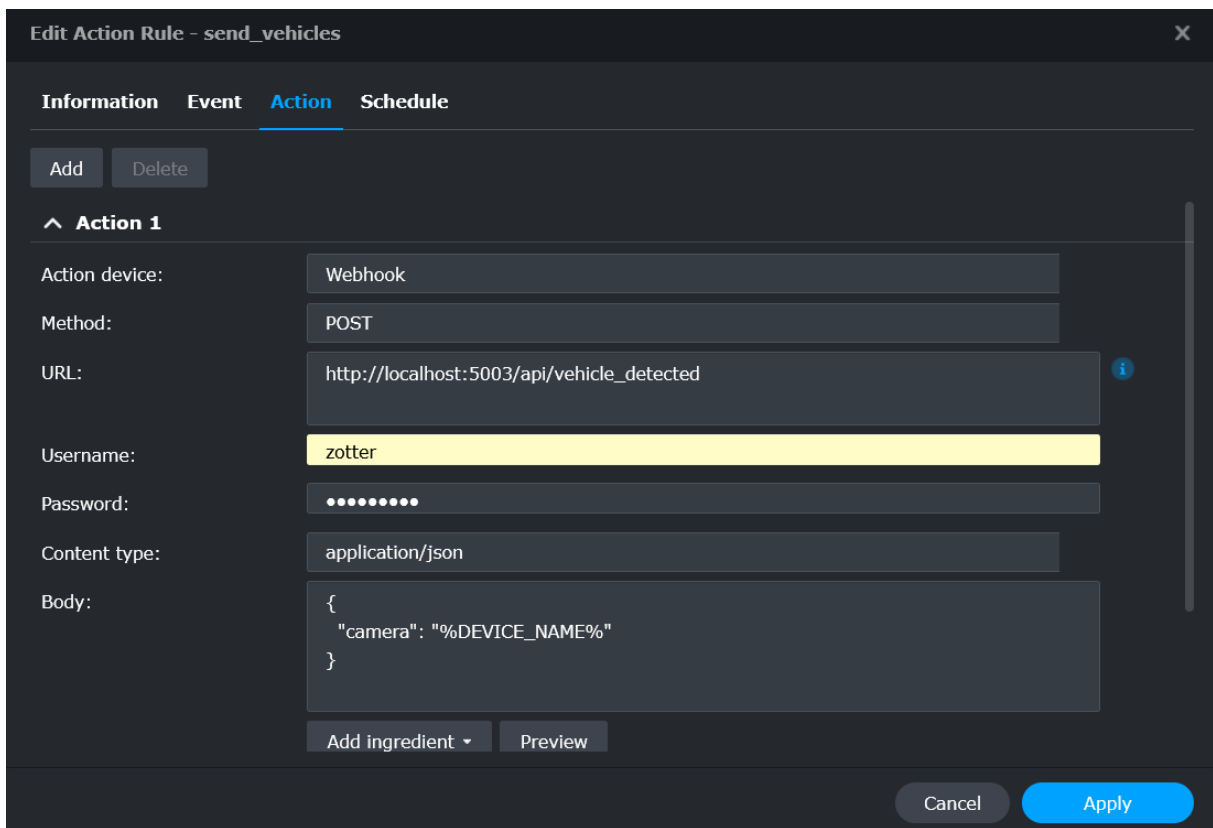


**Bild 18:** Erkennungsbereich im Deep Video Analytics Tool

<sup>2</sup>Der in der Abbildung ersichtliche schwarze Balken in der rechten, oberen Ecke dient dem Datenschutz der auf der anderen Straßenseite befindlichen Familienhäuser und schränkt die Funktion des Systems in keiner Weise ein.

### 5.1.3 Action Rule Konfiguration

Innerhalb der Surveillance Station können sogenannte Action-Rules definiert werden, welche bei Erkennung einer Fahrzeugbewegung im konfigurierten Erkennungsbereich eine zuvor definierte Aktion ausführen. Dieses System wird genutzt, um im Falle einer Erkennung dem Data Collection Service zu signalisieren, dass eine Fahrzeugerkennung stattgefunden hat. Diese Aktionsregel wird aufgerufen, nachdem der zuvor konfigurierte DVA-Task ein Fahrzeug erkennt und ist damit der Startpunkt, für die im Backend stattfindende Verarbeitung der Bilddaten. Wie in Abbildung 19 erkenntlich, ruft diese einen Webhook des Data Collection Service auf; dieser wird im Abschnitt 5.2.2 näher beleuchtet.



**Edit Action Rule - send\_vehicles**

Information Event **Action** Schedule

Add Delete

**^ Action 1**

Action device: Webhook

Method: POST

URL: `http://localhost:5003/api/vehicle_detected`

Username: `zotter`

Password: ••••••••

Content type: `application/json`

Body: 

```
{
  "camera": "%DEVICE_NAME%"
}
```

Add ingredient Preview

Cancel Apply

**Bild 19:** Action-Rule für die Erkennung von Fahrzeugen



## 5.2 Data Collection Service

Der Data Collection Service bildet das funktionale Herzstück des gesamten Systems. Dieser ist für die Entgegennahme von Fahrzeugerkennungseignissen, die Beschaffung von Kamera-Snapshots, die Weiterleitung an die ALPR-Komponente zur Erkennung, sowie die Verarbeitung, Anonymisierung und persistente Speicherung der Ergebnisse verantwortlich.

### 5.2.1 Konzept und Aufbau

Der Service ist als FastAPI-Applikation implementiert und folgt einer modularen Architektur, wobei jeder funktionale Bereich in eigene Klassen, einen so genannten Handler, ausgelagert ist, was eine klare Trennung der Verantwortlichkeiten (Separation of Concerns) ermöglicht:

- **CameraHandler:** Dieser kapselt die gesamte Kommunikation mit der Synology Surveillance Station API (Authentifizierung, Kamera-Verwaltung, Snapshot-Abruf).
- **PlateRecognizerHandler:** Verwaltet die HTTP-Kommunikation mit dem Plate Recognizer SDK-Container.
- **CountryHandler:** Übernimmt die Anreicherung der Erkennungsdaten um regionale Herkunftsinformationen (Bezirkserkennung).
- **DatabaseHandler:** Verantwortlich für die Datenaufbereitung, Anonymisierung und die Speicherung der Erkennungen in der Datenbank.

Die Konfiguration des Services erfolgt über Umgebungsvariablen, welche mittels Pydantic Settings validiert werden [**pydantic\_settings**]. Hierzu zählen unter anderem die Zugangsdaten für die Synology-API, der API-Key für Plate Recognizer, die Datenbankverbindungsparameter sowie ein konfigurierbares Duplikat-Erkennungsintervall.

### 5.2.2 Externer Trigger

Eine HTTP-Webhook ist wie zuvor erwähnt der Auslöser für die Verarbeitung der Bilddaten, welche von der Synology Surveillance Station bei der Erkennung eines Fahrzeugs aufgerufen wird. [**synology\_webhooks**]

Der Webhook-Endpunkt `POST /api/vehicle_detected` erwartet im Request-Body den Namen der auslösenden Kamera und ist durch HTTP Basic Authentication geschützt, wie im Sicherheitskonzept (Abschnitt 4.3.1) erläutert. Bei eingehenden Requests wird zunächst

die Authentifizierung gegen die konfigurierten Synology-Credentials geprüft. Nach erfolgreicher Validierung wird die Verarbeitung als Hintergrundaufgabe (Background Task) gestartet, während der Endpunkt sofort mit dem HTTP-Statuscode 202 Accepted antwortet.

Dieses asynchrone Muster ist essenziell, um die Reaktionsfähigkeit des Systems zu gewährleisten und Blockaden während der Bildverarbeitung zu vermeiden. Da die vollständige Verarbeitungskette, bestehend aus Snapshot-Abruf, ALPR-Analyse und Datenbank-speicherung, mehrere Sekunden in Anspruch nehmen kann, würde eine synchrone Verarbeitung die Surveillance Station blocken und potenzielle Folge-Events verzögern. Durch die Entkopplung mittels Background Tasks wird sichergestellt, dass der Webhook-Endpunkt jederzeit mit minimaler Latenz erreichbar bleibt, und auch bei hoher Fahrzeugfrequenz keine Events aufgrund von Timeouts von der Surveillance Station verworfen werden.

Der Detektionszeitpunkt wird dabei bereits im HTTP-Endpoint erfasst und nicht erst im Background Task. Dies stellt sicher, dass der Zeitstempel den tatsächlichen Zeitpunkt des Webhook-Aufrufs widerspiegelt und nicht den Zeitpunkt der eventuell verzögerten Verarbeitung.

#### Listing 1: Webhook-Endpunkt für Fahrzeugerkennung

```
1 @app.post('/api/vehicle_detected')
2 async def handle_vehicle_detection(
3     request: VehicleDetectionRequest,
4     background_tasks: BackgroundTasks,
5     credentials: HTTPBasicCredentials = Depends(basic_auth),
6 ):
7     if credentials.username != settings.synology_username \
8         or credentials.password != settings.synology_password:
9         raise HTTPException(status_code=401,
10                             detail='Incorrect username or password')
11
12     logger.info(f'Vehicle detected from camera: {request.camera}')
13
14     detection_time = datetime.now()
15     timestamp_str = detection_time.strftime('%Y%m%d_%H%M%S')
16
17     background_tasks.add_task(
18         process_vehicle_detection,
19         request.camera,
20         detection_time,
21     )
22
23     return {'status': 'accepted', 'timestamp': timestamp_str}
```

### 5.2.3 Ablauf der Verarbeitung

Die im Anhang angeführte Darstellung (Abb. ??) zeigt den vollständigen Ablauf von der Webhook-Auslösung bis zur Speicherung in der Datenbank als Sequenzdiagramm. Jede Erkennung durchläuft nach dem Webhook-Eingang die folgenden Phasen, welche in den nächsten Unterkapiteln im Detail erläutert werden: Kamera-Anbindung, ALPR-Aufruf, Datenanreicherung, Anonymisierung und Speicherung mit Duplikatprüfung.

### 5.2.4 Kamera-Anbindung (CameraHandler)

Die Kommunikation mit der Synology Surveillance Station erfolgt über die dezidierte Web-API dieser und ist in der CameraHandler-Klasse gekapselt [**synology\_api**]. Der Ablauf der Kamera-Interaktion gliedert sich in drei Schritte:

**1. Authentifizierung** Zunächst authentifiziert sich der Service über den API-Endpunkt `SYNO.API.Auth` und erhält eine Session-ID (SID), die für alle nachfolgenden API-Aufrufe als Identifikation dient.

**2. Kameraliste und Identifikation** Im zweiten Schritt wird die Liste aller im Surveillance-Station-Netzwerk registrierten Kameras über den `SYNO.SurveillanceStation.Camera`-Endpunkt abgerufen. Aus dieser Liste wird die Kamera anhand des vom Webhook übermittelten Namens identifiziert und die zugehörige Kamera-ID extrahiert.

**3. Snapshot-Abruf** Mit der ermittelten Kamera-ID wird über die `GetSnapshot`-Methode der Synology-API ein aktueller Kamera-Snapshot, also das Kamerabild zum Zeitpunkt der Anfrage, im JPEG-Format angefordert. Die Bilddaten werden als Byte-Stream im Arbeitsspeicher gehalten, ohne auf die Festplatte geschrieben zu werden, um personenbezogene Bilddaten nicht persistent zu speichern. Optional kann im Debug-Modus, welcher über eine Umgebungsvariable aktiviert wird, das Bild zu Diagnosezwecken auf die Festplatte geschrieben werden.

Jeder der drei Schritte ist durch spezifische Exceptions (`AuthenticationError`, `CameraDataError`, `SnapshotError`) abgesichert, die im Fehlerfall die Verarbeitungskette kontrolliert abbrechen.

### 5.2.5 Plate Recognizer Integration (PlateRecognizerHandler)

Die Bilddaten des Kamera-Snapshots werden, zur Kennzeichenerkennung, an den lokal betriebenen Plate Recognizer SDK-Container gesendet. Die `PlateRecognizerHandler`-

Klasse implementiert diesen HTTP-Aufruf mit einer Retry-Strategie mittels der tenacity-Bibliothek [**tenacity**]:

**Listing 2:** Plate Recognizer Integration mit Retry-Strategie

```

1 class PlateRecognizerHandler:
2     @retry(
3         wait=wait_fixed(1),
4         stop=stop_after_attempt(5),
5         retry=retry_if_exception_type(requests.HTTPError),
6         reraise=True,
7     )
8     def send_to_api(self, api_key: str, image_data: bytes,
9                     camera_name: str, service_url: str) -> Any:
10        image_buffer = BytesIO(image_data)
11        response = requests.post(
12            f'{service_url}/v1/plate-reader/',
13            data={
14                'camera_id': camera_name,
15                'regions': ['at', 'hu', 'si', 'de'],
16                'mmc': 'true',
17                'direction': 'true',
18            },
19            files={'upload': image_buffer},
20            headers={'Authorization': f'Token {api_key}'},
21            timeout=15,
22        )
23        response.raise_for_status()
24        return response.json()

```

Der `@retry`-Dekorator sorgt dafür, dass bei HTTP-Fehlern (etwa bei temporären Überlastungen oder Blockaden des ALPR-Containers) bis zu fünf Wiederholungsversuche im Abstand von einer Sekunde unternommen werden. Dies erhöht die Robustheit des Systems, da kurzzeitige Ausfälle der ALPR-Komponente nicht zum Verlust von Erkennungen führen. Der Parameter `regions` schränkt die Erkennung auf die relevanten Regionen ein, für diese Diplomarbeit wurden aufgrund der Lage der Zotter Schokolade GmbH, Österreich, Ungarn, Slowenien und Deutschland gewählt. Dies verbessert die Erkennungsgenauigkeit, da das Modell die länderspezifischen Kennzeichenformate dieser Regionen priorisiert [**platerrecognizer\_api**]. Über den Parameter `mmc` (Make, Model, Color) wird die erweiterte Fahrzeugerkennung aktiviert, durch `direction` wird die Fahrtrichtung anhand der Fahrzeugorientierung angefordert. Die Bilddaten werden als `BytesIO`-Objekt übergeben, also als im Arbeitsspeicher gehaltener Byte-Stream.

### 5.2.6 Datenverarbeitung und Enrichment-Schleife

Nach dem Empfang der JSON-Antwort von Plate Recognizer durchlaufen die Erkennungsdaten einen mehrstufigen Verarbeitungs- und Anreicherungsprozess, bis diese final und persistent in der Datenbank gespeichert werden. Die Plate-Recognizer-API kann in einem einzelnen Bild mehrere Kennzeichen erkennen, weshalb die Antwort eine Liste von Ergebnissen enthält, welche jeweils einzeln in einer Schleife verarbeitet werden.

**Datenextraktion** Im ersten Schritt werden die relevanten Felder aus der Antwort der Plate-Recognizer-API extrahiert und in ein Pydantic-Schema überführt. Die API liefert den Konfidenzwert jeder Erkennung als Float-Zahl zwischen 0 und 1 zurück, welcher für die Speicherung mit 1000 multipliziert und als ganzzahliger Wert abgelegt wird. Die Fahrzeugorientierung wird aus der API-Antwort als Enum (FRONT oder REAR) abgebildet, wobei FRONT einer Ausfahrt und REAR einer Einfahrt entspricht, da die Kamera in Richtung des Parkplatzes filmt.

**Bezirkserkennung (CountryHandler)** Die Bezirkserkennung ist ein zentraler Bestandteil der Datenanreicherung und ermöglicht es, das Herkunftsbundesland bzw. den Herkunftsbezirk eines Fahrzeugs aus dem Kennzeichen abzuleiten. Die Implementierung nutzt hierfür eine JSON-Datei (`municipalities.json`), welche Bezirkskürzel für österreichische [**oesterreich\_kfz**] und slowenische [**wiki\_slovenia\_plates**] Kennzeichen deren vollen Namen und Bundesländern zuordnet. Diese Zuordnungstabellen werden beim Start des Services einmalig in den Arbeitsspeicher geladen und als Dictionary-Lookup verwendet.

Der Erkennungsalgorithmus für österreichische Kennzeichen versucht zunächst, die ersten zwei Zeichen des Kennzeichens als Bezirkskürzel zu interpretieren. Da einige österreichische Bezirke einbuchstabige Kürzel verwenden (wie z.B. „W“ für Wien oder „G“ für Graz), wird bei fehlgeschlagenem Zwei-Buchstaben-Lookup ein Fallback auf einbuchstabige Kürzel durchgeführt.

Für slowenische Kennzeichen wird ein ähnlicher Ansatz verfolgt, mit dem zusätzlichen Aspekt, dass der CountryHandler hier auch als Korrekturmechanismus dient: Wird ein Kennzeichen von Plate Recognizer mit dem Ländercode `unknown` klassifiziert, der Kennzeichen-Präfix aber als slowenischer Bezirkscode erkannt, wird der Ländercode automatisch auf `si` korrigiert. Diese Korrektur adressiert eine Limitierung der ALPR-Engine bei slowenischen Kennzeichen, welche aufgrund visueller Ähnlichkeiten zu anderen Formaten oft unerkannt bleiben.

**Anonymisierung (Hashing)** Nach der Datenanreicherung folgt die Anonymisierung des Klartext-Kennzeichens. Wie im Sicherheitskonzept (Abschnitt 4.3.3) beschrieben, wird das Kennzeichen mittels SHA-256 irreversibel gehasht. Das Kennzeichen wird vor dem Hashing normalisiert (Trimmen von Leerzeichen, Konvertierung in Kleinbuchstaben), um sicherzustellen, dass identische Kennzeichen unabhängig von Schreibweisenunterschieden in der OCR-Erkennung denselben Hash erzeugen. Der resultierende 32-Byte-Hash wird in Binärdaten in der Datenbank gespeichert.

### 5.2.7 Duplikatprüfung und Speicherung

Da die Surveillance Station bei kontinuierlicher Fahrzeugbewegung im Erkennungsbereich mehrere Webhooks in kurzem Abstand auslösen kann, implementiert der Service einen zeitbasierten Duplikatsfilter. Vor dem Einfügen einer neuen Erkennung prüft die unten angeführte Methode `check_for_duplicates`, ob in der Datenbank bereits eine Erkennung mit dem gleichen Kennzeichen-Hash innerhalb eines konfigurierbaren Zeitintervalls existiert.

**Listing 3:** Zeitfensterbasierte Duplikatserkennung

```
1 def check_for_duplicates(self, db: Session,
2     observation: VehicleObservationCreate,
3     current_detection_time: datetime,
4     interval: timedelta) -> bool:
5     start_time = current_detection_time - interval
6     stmt = select(VehicleObservation).where(
7         and_(
8             VehicleObservation.plate_hash == observation.plate_hash,
9             VehicleObservation.timestamp >= start_time,
10            VehicleObservation.timestamp <= current_detection_time,
11        )
12    )
13    duplicate_observation = db.execute(stmt).scalars().first()
14    return duplicate_observation is not None
```

Das Intervall ist über die Umgebungsvariable `INTERVAL_SECONDS` (Standardwert: 60 Sekunden) konfigurierbar. Im Praxisbetrieb hat sich dieser Wert als geeignet erwiesen, um einerseits Mehrfacherkennungen desselben Fahrzeugs bei langsamer Durchfahrt zu eliminieren, andererseits aber Fahrzeuge, welche innerhalb kurzer Zeit tatsächlich ein- und ausfahren, korrekt als separate Ereignisse zu erfassen. Nur wenn kein Duplikat gefunden wird, wird der anonymisierte und angereicherte Datensatz in die Datenbank eingefügt.

Die volle Methode für die Hintergrundverarbeitung von Erkennungen ist im Anhang einsehbar.

## 5.3 Notification Service

Der Notification Service ist ein FastAPI-Microservice, der die Verwaltung von Benutzerpräferenzen und den Versand von E-Mail-Benachrichtigungen übernimmt.

### 5.3.1 Architektur und API-Design

Der Service stellt zwei primäre API-Routen bereit, welche unter dem /api-Präfix gruppiert sind. Alle Endpunkte sind durch API-Key-Authentication im Authorization-Header geschützt, wie im Sicherheitskonzept erläutert wurde.

**Benutzerpräferenzen (/api/user\_preferences)** Diese Route stellt eine vollständige CRUD-Schnittstelle (Create, Read, Update, Delete) für die Verwaltung von Benutzerpräferenzen bereit. Benutzer können über REST-Endpunkte erstellt, abgefragt, aktualisiert und gelöscht werden. Jeder Benutzer verfügt über zwei unabhängige Flags: `receive_alerts` für Echtzeit-Benachrichtigungen und `receive_updates` für periodische Zusammenfassungen.

**Benachrichtigungsversand (/api/notifications)** Der Benachrichtigungsendpunkt POST `/api/notifications/send` nimmt einen Request mit dem Benachrichtigungstyp (`alert` oder `update`), Betreff, Inhalt und optionaler Empfängerliste mit Email-Adressen entgegen. Wenn der Request eine explizite Empfängerliste enthält, werden ausschließlich diese Adressen verwendet. Andernfalls werden alle Benutzer aus der Datenbank anhand des jeweiligen Präferenz-Flags (`receive_alerts` bzw. `receive_updates`) gefiltert.

### 5.3.2 E-Mail-Versand (EmailHandler)

Der eigentliche E-Mail-Versand erfolgt über den `EmailHandler`, welcher E-Mails über einen SMTP-Relay-Server versendet [`mailtrap_python_email`]. Der SMTP-Server des Unternehmens wird als Relay für den internen Versand SMTP (Simple Mail Transfer Protocol) genutzt, um zu verhindern, dass die versendeten Mails aufgrund von Spam-Verdacht nicht korrekt zugestellt werden. [`mailjet_smtp_relay`]

Die Implementierung unterstützt sowohl Plaintext- als auch HTML-formatierte E-Mails. Der Versand an mehrere Empfänger erfolgt über die `send_bulk_email`-Methode, welche für jeden Empfänger eine eigene SMTP-Verbindung aufbaut. Fehlgeschlagene Zustellungen an einzelne Empfänger führen nicht zum Abbruch des gesamten Versandvorgangs, somit wird sichergestellt, dass ein ungültiger Empfänger nicht den Versand an alle anderen blockiert.

## 5.4 Grafana

Als Visualisierungsplattform für die gesammelten Fahrzeugerkennungen dient Grafana, ein Open-Source-Tool für Datenvisualisierung und Monitoring. **[grafana]** Grafana wird als eigener Container im Docker-Compose-Stack betrieben und verbindet sich lesend mit der PostgreSQL-Datenbank.

### 5.4.1 Architektur und Provisioning

Die Grafana-Konfiguration folgt einem Provisioning-Ansatz, bei dem Datenquellen und Dashboards nicht manuell über die Benutzeroberfläche, sondern deklarativ über Konfigurationsdateien definiert werden **[grafana\_provisioning]**. Dies hat den Vorteil, dass die gesamte Dashboard-Konfiguration im Repository passiert und beim Neustart des Containers automatisch wiederhergestellt wird.

Die Konfiguration gliedert sich in drei Bereiche:

**1. grafana.ini:** Grundkonfiguration des Grafana-Servers, unter anderem HTTP-Port, Sicherheitseinstellungen und der Pfad zum Standard-Dashboard, das beim Login automatisch geladen wird **[grafana\_configure]**.

**2. Datasource-Provisioning (datasource.yaml):** Definiert die Verbindung zur PostgreSQL-Datenbank. Grafana verbindet sich über den `analytics_user`, der wie im Sicherheitskonzept beschrieben nur Leserechte auf das Ingestion-Schema besitzt. Die Verbindungsparameter (Host, Datenbankname, Credentials) werden über Umgebungsvariablen gesetzt.

**3. Dashboard-Provisioning (dashboard-provider.yaml und Dashboard-JSON):** Das Dashboard wird als JSON-Datei bereitgestellt und beim Start, wie oben erwähnt, automatisch geladen. Änderungen am Dashboard werden direkt in der JSON-Datei vorgenommen.



### 5.4.2 Dashboard-Aufbau und Darstellungen

Das Dashboard bietet einen umfassenden Überblick über die gesammelten Fahrzeugdaten. Alle Panels verwenden direkte SQL-Abfragen auf das Ingestion-Schema und unterstützen die dynamische Filterung nach Fahrzeugtyp sowie einen frei wählbaren Zeitraum.

Das Dashboard umfasst folgende Darstellungen:

- **Zeitreihendiagramm:** Verlauf der einzigartigen Fahrzeuge über die Zeit, mit dynamischer Aggregation je nach gewähltem Zeitraum.
- **Kennzahlen-Panels:** Acht Statistik-Boxen mit den wichtigsten Metriken, darunter einzigartige Fahrzeuge (Gesamt, Heute, Tagesdurchschnitt), Gesamterkennungen, Anzahl verschiedener Herkunftsländer und Fahrzeugtypen.
- **Länderverteilung:** Kreisdiagramm der Top-10-Herkunftsländer mit prozentualen Anteilen.
- **Fahrzeugtypverteilung:** Horizontales Balkendiagramm der erkannten Fahrzeugkategorien.
- **Verkehrsdichte nach Tageszeit:** Balkendiagramm mit stündlicher Aggregation zur Identifikation von Stoßzeiten.
- **Fahrzeugdetails:** Tabellen der häufigsten Fahrzeugmarken und -modelle sowie ein Balkendiagramm der Fahrzeugfarben, basierend auf der MMC-Erkennung von Plate Recognizer.
- **Regionale Herkunft:** Tabelle der österreichischen Bezirkskürzel zur visuellen Einordnung der häufigsten Herkunftsregionen.

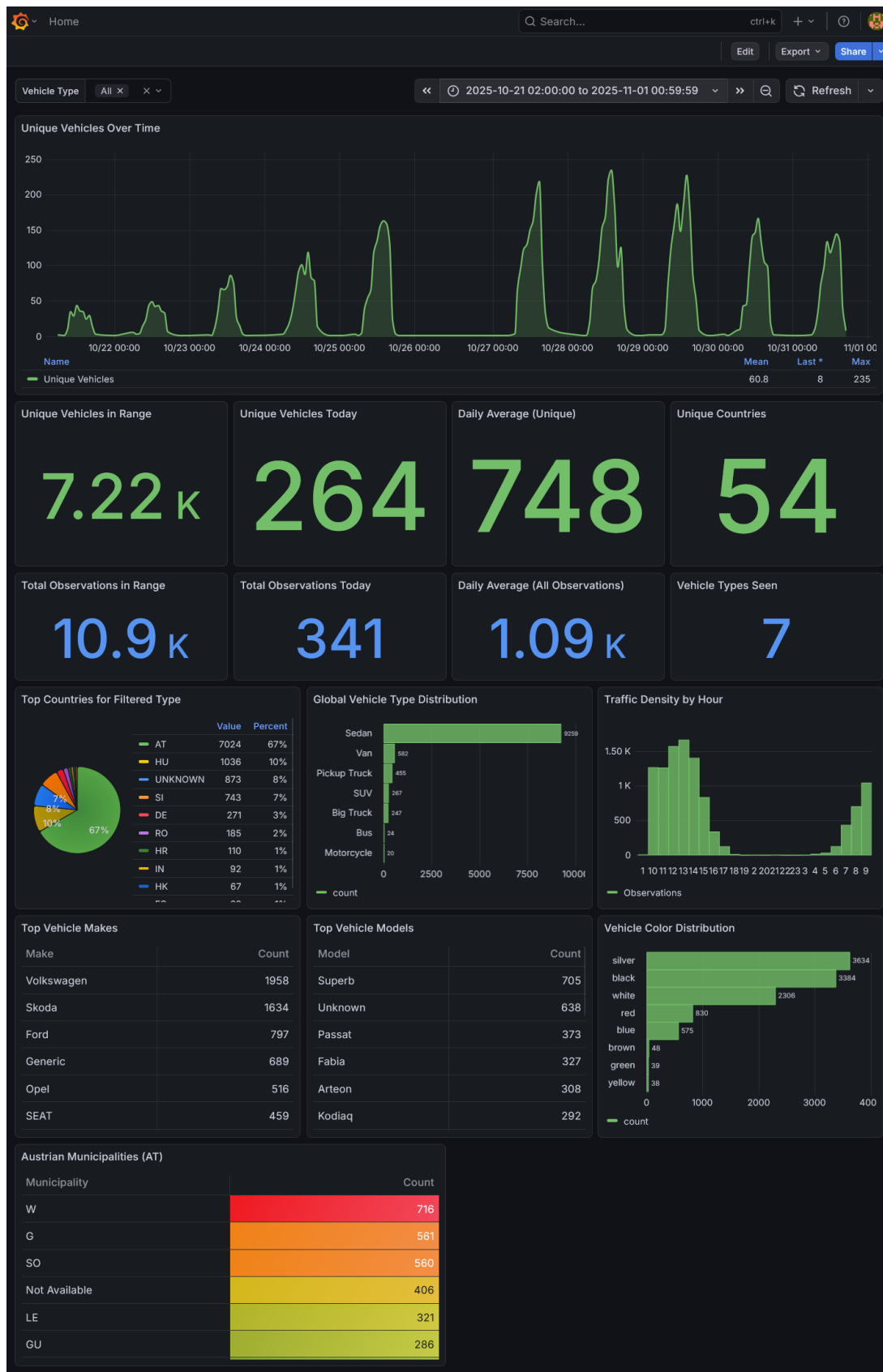


Bild 20: Grafana Dashboard

## 6 Infrastruktur, Deployment und Betrieb

Dieses Kapitel behandelt die infrastrukturellen Komponenten und Prozesse dieser Diplomarbeit. Erläutert werden unter anderem der Deployment-Prozess sowie die im laufenden Betrieb relevanten Aspekte wie Monitoring und Logging, Datenbank-Management und Backup-Strategien.

### 6.1 Container-Orchestrierung

Die gesamte Applikation wird, mittels Docker Compose orchestriert, wobei zwei separate Konfigurationsdateien für die Entwicklungs- und Produktionsumgebungen benutzt werden. Docker Compose nutzt, wie bereits im entsprechenden Kapitel erwähnt, YAML-Dateien, welche zur Kennung mit den Endungen `.prod.yaml` und `.dev.yaml` versehen wurden. Beide Konfigurationen definieren dieselben Services, unterscheiden sich jedoch grundlegend in der Art und Weise, wie diese für Docker zum Starten bereitgestellt werden.

#### 6.1.1 Entwicklungsumgebung

In der Entwicklungsumgebung werden alle Services lokal aus ihren jeweiligen Dockerfiles gebaut. Ein zentrales Merkmal ist die Verwendung von Volume-Mounts, welche lokale Dateiverzeichnisse direkt in die Container einbinden:

**Listing 4:** Volume Mount Entwicklungs-Compose-Konfiguration

```
1 data-collection-service:
2   build:
3     context: ./services/data-collection-service
4     dockerfile: Dockerfile
5   volumes:
6     - ./services/data-collection-service/src:/app/src:rw
```

Durch diese Konfiguration werden Codeänderungen sofort im laufenden Container wirksam (Hot-Reload), ohne dass der Container neu gebaut werden muss. Dies beschleunigt den Entwicklungszyklus erheblich, da Änderungen am Quellcode schnell in der Docker-Umgebung getestet werden können.

#### 6.1.2 Produktionsumgebung

Die Produktionskonfiguration verwendet hingegen vorgefertigte Docker-Images, welche über eine Docker-Registry heruntergeladen werden:

**Listing 5:** Image Produktions-Compose-Konfiguration

```
1 data-collection-service:
2   image: julianbierbaum/license-plate-system:data-collection-service
3   restart: unless-stopped
```

Die Volume-Mounts für den Quellcode entfallen hier vollständig, da dieser direkt in die zuvor gebauten Images eingebettet wurde. Eine wichtige Ausnahme hierbei ist allerdings das Volume der Datenbank, welches für den Bestand der Daten natürlich weiterhin außerhalb des Images gespeichert wird. Dies bedeutet, dass diese Images vollständig eigenständig sind und alle Abhängigkeiten sowie den Applikationscode bereits enthalten.

Für das Deployment auf der Produktionsumgebung werden daher ausschließlich die Docker Compose-Datei und die Umgebungsvariablen zur Konfiguration benötigt, ein Klonen des Quellcode-Repositories ist nicht erforderlich. Dies ist für die Nutzung von Portainer wichtig, ein Tool, welches später in diesem Kapitel näher beleuchtet wird. Zusätzlich wurde die Restart-Policy strikter gesetzt, um sicherzustellen, dass Container nach einem Absturz oder einem Neustart des Host-Systems automatisch wieder hochgefahren werden.

### 6.1.3 Startabhängigkeiten und Healthchecks

Ein wesentlicher Aspekt der Orchestrierung ist die korrekte Startreihenfolge der Services. Da alle Applikationsservices eine funktionierende Datenbank voraussetzen, wird ein spezieller Datenbank-Prestart-Container eingesetzt, welcher die Datenbank initial konfiguriert. Erst wenn dieser Container erfolgreich durchgelaufen ist, werden die abhängigen Services gestartet:

**Listing 6:** Startabhängigkeiten Datenbank Prestart

```
1 depends_on:
2   db-prestart:
3     condition: service_completed_successfully
```

Der Prestart-Service wartet selbst wiederum auf den PostgreSQL-Container, welcher über einen so genannten Healthcheck seine Bereitschaft signalisiert. Zusätzlich verfügen die Backend-Services und der Plate Recognizer-Container über eigene Healthchecks, mit welchen, über definierte HTTP-Endpunkte der aktuellen Zustand eines Services überwacht werden kann.

## 6.2 CI/CD Pipelines und Deployment

CI/CD (Continuous Integration / Continuous Delivery) bezeichnet die Praxis, Codeänderungen automatisiert zu testen, bauen und bereitzustellen [**redhat\_cicd**]. Continuous Integration stellt sicher, dass Codeänderungen automatisch auf Fehler geprüft werden, bevor diese in den Hauptbranch des Code-Repositories aufgenommen werden. Continuous Delivery automatisiert darauf aufbauend den Build- und Bereitstellungsprozess, sodass neue Versionen automatisch auf dem Docker-Registry als Images zur Verfügung gestellt werden. Die Umsetzung dieser Prinzipien erfolgt im Projekt über GitHub Actions, ein in GitHub integriertes Automatisierungstool, mit welchem Workflows zum Bauen, Testen und Bereitstellen von Software direkt im Repository definiert werden können [**github\_actions**]. Diese Prozesse werden durch Ereignisse wie Code-Pushes oder Merges automatisch ausgelöst und führen die konfigurierten Schritte auf virtuellen Servern aus.

### 6.2.1 Build und Test Pipeline

Die primäre CI/CD-Pipeline wird bei jedem Push auf einen beliebigen Branch ausgelöst und besteht aus zwei sequenziellen Phasen:

**Phase 1: Testing und Linting** In der ersten Phase werden für jeden Python-Service automatisiert drei Prüfungen durchgeführt:

1. Formatierung (Ruff Format [**ruff\_formatter**]): Überprüfung des Codes auf Einhaltung der definierten Formatierungsregeln.
2. Linting (Ruff Check [**ruff\_linter**]): Statische Codeanalyse zur Erkennung von potenziellen Fehlern und Stilabweichungen.
3. Tests (Pytest mit Coverage): Ausführung aller Test-Cases, welche mit einer Codeabdeckungsanalyse kombiniert werden.

Alle Tests werden innerhalb von Docker-Containern ausgeführt, um eine konsistente und reproduzierbare Testumgebung zu gewährleisten, welche der Produktionsumgebung möglichst nahekommt.

**Phase 2: Build und Push** Wenn die Testphase erfolgreich abgeschlossen wurde und der Push auf dem Haupt-Branch erfolgt, wird die zweite Phase ausgelöst. Diese nutzt ein Build-Script, welches automatisiert alle Docker-Images baut und auf das definierte Registry pusht.

Listing 7: Auszug Build-Script build.sh

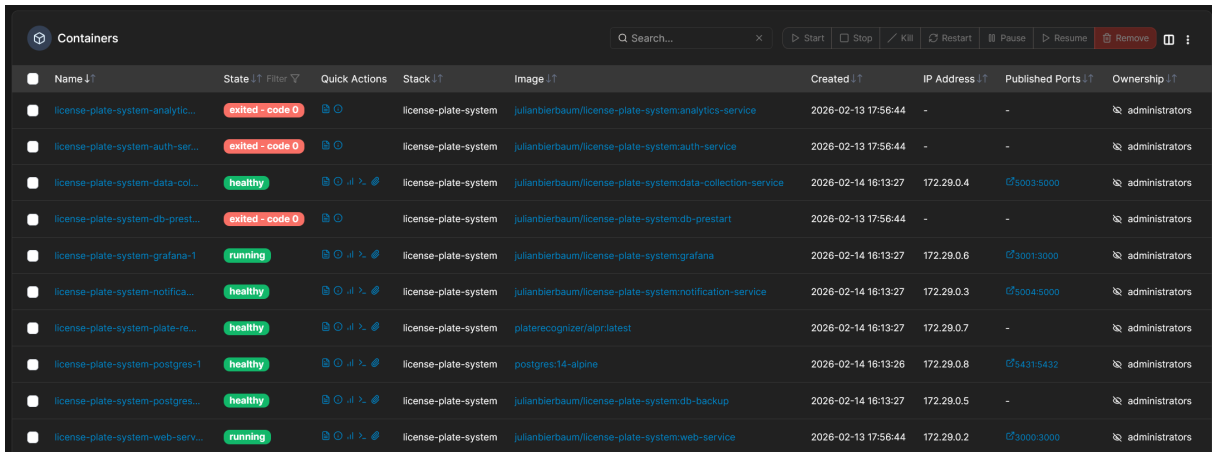
```
1 # Build all services
2 for SERVICE_DIR in services/*/; do
3     SERVICE_NAME=$(basename "$SERVICE_DIR")
4     echo "--- Building ${SERVICE_NAME} ---"
5
6     # Check Python lock file
7     if [ -f "${SERVICE_DIR}pyproject.toml" ] && [ ! -f "${SERVICE_DIR}uv.lock" ];
8     then
9         echo "Error: ${SERVICE_DIR}uv.lock not found. Run uv sync first."
10        exit 1
11    fi
12
13    # Check Node.js lock file
14    if [ -f "${SERVICE_DIR}package.json" ] && [ ! -f "${SERVICE_DIR}package-lock.json" ]; then
15        echo "Error: ${SERVICE_DIR}package-lock.json not found. Run npm install first."
16        exit 1
17    fi
18
19    docker build -t "${DOCKER_REGISTRY}:${SERVICE_NAME}" "$SERVICE_DIR"
20    docker push "${DOCKER_REGISTRY}:${SERVICE_NAME}"
21 done
```

Dieses iteriert, wie im Code-Ausschnitt zu sehen ist, über alle Service-Verzeichnisse und identifiziert dabei Services anhand ihrer Lock-Files. Da für die Python-Services UV als Package Manager benutzt wurde, dient das `uv.lock`-File als Identifikationsmerkmal für diese. Frontend-Services werden mittels des `package-lock.json`-Files identifiziert und alle anderen Services, welche nicht im Service-Verzeichnis liegen, werden separat aufgerufen. Das vollständige `build.sh`-Script ist im Anhang einsehbar.

## 6.2.2 Deployment mit Portainer

Deployment auf der Produktionsumgebung erfolgt über die Plattform Portainer, ein Container Management-Tool mit Weboberfläche **[portainer]**. Portainer wurde auf dem Synology-NAS installiert und ermöglicht das Verwalten der Docker-Container direkt über einen Browser.

Für das Deployment wird in Portainer ein Stack erstellt, welcher die Produktions-Compose-Datei sowie die Datei mit den Umgebungsvariablen enthält. Bei Aktualisierungen werden die neuesten Images aus dem Docker-Registry heruntergeladen, woraufhin die betroffenen Container automatisch neu erstellt werden, ohne dass die Datenhaltung betroffen ist.



| Name                             | State           | Quick Actions | Stack                | Image                                                       | Created             | IP Address | Published Ports | Ownership      |
|----------------------------------|-----------------|---------------|----------------------|-------------------------------------------------------------|---------------------|------------|-----------------|----------------|
| license-plate-system-analytic... | exited - code 0 |               | license-plate-system | julianbierbaum/license-plate-system:analytics-service       | 2026-02-13 17:56:44 | -          | -               | administrators |
| license-plate-system-auth-ser... | exited - code 0 |               | license-plate-system | julianbierbaum/license-plate-system:auth-service            | 2026-02-13 17:56:44 | -          | -               | administrators |
| license-plate-system-data-col... | healthy         |               | license-plate-system | julianbierbaum/license-plate-system:data-collection-service | 2026-02-14 16:13:27 | 172.29.0.4 | 05003:5000      | administrators |
| license-plate-system-db-prest... | exited - code 0 |               | license-plate-system | julianbierbaum/license-plate-system:db-prestart             | 2026-02-13 17:56:44 | -          | -               | administrators |
| license-plate-system-grafana-1   | running         |               | license-plate-system | julianbierbaum/license-plate-system:grafana                 | 2026-02-14 16:13:27 | 172.29.0.6 | 03001:3000      | administrators |
| license-plate-system-notifica... | healthy         |               | license-plate-system | julianbierbaum/license-plate-system:notification-service    | 2026-02-14 16:13:27 | 172.29.0.3 | 05004:5000      | administrators |
| license-plate-system-plate-re... | healthy         |               | license-plate-system | platerecognizer/alprlatest                                  | 2026-02-14 16:13:27 | 172.29.0.7 | -               | administrators |
| license-plate-system-postgres-1  | healthy         |               | license-plate-system | postgres:14-alpine                                          | 2026-02-14 16:13:26 | 172.29.0.8 | 05431:5432      | administrators |
| license-plate-system-postgres... | healthy         |               | license-plate-system | julianbierbaum/license-plate-system:db-backup               | 2026-02-14 16:13:27 | 172.29.0.5 | -               | administrators |
| license-plate-system-web-serv... | running         |               | license-plate-system | julianbierbaum/license-plate-system:web-service             | 2026-02-13 17:56:44 | 172.29.0.2 | 03000:3000      | administrators |

Bild 21: Laufender Container-Stack in Portainer

## 6.3 Monitoring und Logging

### 6.3.1 Healthchecks

Wie bereits erwähnt verfügen die Services über Healthchecks, welche von Docker Compose in regelmäßigen Intervallen (alle 5 Sekunden) ausgeführt werden und primär dazu dienen, die Verfügbarkeit der Services zu überwachen. Die Methode des Healthchecks variiert je nach Service-Art:

- Backend-Services (Data Collection, Notification): HTTP-Request an den `/health`-Endpunkt des jeweiligen Services.
- PostgreSQL-Datenbank: Verwendung des nativen `pg_isready`-Kommandos zur Überprüfung der Datenbankbereitschaft.
- Plate Recognizer: HTTP-Request an den Root-Endpunkt des ALPR-Containers.
- Backup-Service: Überprüfung, ob der Cron-Daemon (`crond`) als Prozess läuft.

Der Status der Healthchecks ist über Portainer einsehbar und ermöglicht somit eine zentrale Überwachung des Status aller Container.

### 6.3.2 Logging

Alle Services wurden so konfiguriert, dass Log-Ausgaben direkt auf die Konsole geschrieben werden (Console-Logging). Das Log-Level ist über eine Umgebungsvariable konfigurierbar und wird in der Produktionsumgebung typischerweise auf `INFO` gesetzt, während in der Entwicklungsumgebung `DEBUG` als Standard dient.

Die Logs sind über Docker und Portainer einsehbar. Durch die Verwendung des Python Logging-Moduls in allen Backend-Services wird ein einheitliches Log-Format erzeugt, welches Zeitstempel, Log-Level und den Namen des Services enthält, auf welchem der Log erstellt wurde.

**Listing 8:** Beispielhafte Log-Ausgabe des Data Collection Services

```
1 2026-02-17 15:55:24,782 - data-collection-service - INFO - Vehicle detected from  
   camera: BC500-001  
2 2026-02-17 15:55:26,461 - data-collection-service - INFO - No vehicle observations  
   found in reader result.
```

Des Weiteren könnte dieses Modul zukünftig ohne großen Mehraufwand erweitert werden, um etwa gleichzeitig mit dem Console-Log in zentrale Log-Files zu schreiben.

## 6.4 Datenbank-Management

### 6.4.1 Initiale Einrichtung

Die initiale Einrichtung der Datenbank erfolgt automatisch beim ersten Start des Systems über den DB-Prestart-Container. Dieser führt ein Entrypoint-Script aus, das zwei Aufgaben erfüllt: Zunächst führt dieser ein Init-Script aus. Dieses SQL-Script erstellt die spezifischen Datenbank-Schemas und -Benutzer der Services, welche bereits im Kapitel 4.2.1 erläutert wurden. Zudem werden die granularen Berechtigungen vergeben, wie sie im Sicherheitskonzept (Abschnitt 4.3.2) definiert wurden, beispielsweise erhält der Analytics-Service-User nur Leserechte auf das Ingestion-Schema.

Die zweite Aufgabe dieses Services ist die Ausführung der Datenbankmigrationen unter Benutzung von Alembic. Nach der zuvor erwähnten Schema-Erstellung werden automatisch alle ausstehenden Migrationen angewendet, um die Tabellen in den aktuellen Zustand zu bringen. Durch diese automatisierte Einrichtung entfällt jegliche manuelle Datenbankadministration beim initialen Deployment oder beim Neuaufsetzen des Systems.

Das init.sql-Script ist im Anhang angeführt.

### 6.4.2 Schema-Migrationen mit Alembic

Für die Versionierung des Datenbankschemas wird Alembic eingesetzt, ein Migrationswerkzeug für SQLAlchemy-basierte Anwendungen.

Der DB-Prestart-Container kopiert die SQLAlchemy-Modelle der einzelnen Services in den eigenen Build-Kontext und kann somit das Schema aller Services zentral verwalten. [[alembic\\_monorepo](#)]



Diese sind mittels Volumes aus den Verzeichnissen der Backend-Services an den Prestart-Container angebunden.

**Listing 9:** Dockerfile COPY Anweisungen

```
1 COPY services/data-collection-service/src/ /app/data_collection/src/
2 COPY services/notification-service/src/ /app/notification/src/
3 # COPY services/analytics-service/src/ /app/analytics/src/
```

Die Alembic-Konfiguration importiert die Base-Klassen der Backend-Services im Environment-Python-File:

**Listing 10:** Alembic target\_metadata

```
1 # Import Base objects
2 from data_collection.src.models.base import IngestionBase
3 from notification.src.models.base import NotificationBase
4 # from analytics.src.models.base import AnalyticsBase
5
6 target_metadata = [
7     IngestionBase.metadata,
8     NotificationBase.metadata,
9     # AnalyticsBase.metadata,
10 ]
```

Durch diesen zentralisierten Ansatz können Migrationen generiert werden, welche über alle Schemas hinweg konsistent sind. Die Migrationen werden wie bei Alembic üblich versioniert und mit dem Repository eingchecked, wobei jede Migration durch eine Python-Datei repräsentiert wird, welche sowohl die Up- als auch Downgrade-Funktion enthält, um Schema-Änderungen vorwärts und rückwärts anwenden zu können.

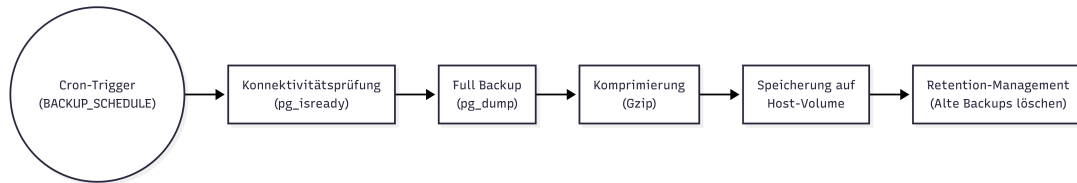
## 6.5 Backup-Strategie

Die Backup-Strategie des Systems umfasst sowohl automatisierte als auch manuelle Sicherungsmechanismen und einen manuellen Wiederherstellungsprozess.

### 6.5.1 Automatisiertes Backup

Für die automatisierte Sicherung der PostgreSQL-Datenbank wird ein dedizierter Backup-Container eingesetzt, welcher als separater Service im Docker Compose-Stack definiert wurde. Dieser Container basiert auf dem offiziellen PostgreSQL-Alpine-Image und nutzt den Cron-Daemon zur zeitgesteuerten Ausführung des Backup-Skripts.

Der Backup-Zeitplan wird über die Umgebungsvariable `BACKUP_SCHEDULE` im Cron-Format konfiguriert (In diesem Fall etwa `0 2 * * *` für ein tägliches Backup um 02:00 Uhr [**cron\_guru**]). Falls die Variable nicht gesetzt wurde, werden automatische Backups deaktiviert und der Container wechselt in einen Idle-Zustand.



**Bild 22:** Backup Flow

Der Cron-Daemon löst das Backup-Script zum konfigurierten Zeitpunkt aus. Dieses wartet zunächst auf die Erreichbarkeit der Datenbank, erstellt dann einen vollständigen Datenbank-Dump mittels `pg_dump` [**postgres\_pgdump**], komprimiert anschließend diesen mit Gzip und schreibt die entstandene Backup-Datei auf ein gemountetes Host-Volume. Abschließend werden Backups, die älter als die konfigurierte Aufbewahrungsdauer (standardmäßig 30 Tage) sind, automatisch gelöscht.

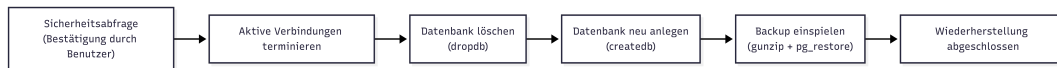
In dieser Implementierung wird bei jedem Durchlauf ein vollständiges Datenbank-Backup (Full Backup) erstellt, anstatt auf inkrementelle oder differentielle Strategien zurückzugreifen. Wie im Kapitel zur Datenbankarchitektur (4.2.3) errechnet, beträgt die Größe eines komprimierten Full-Backups selbst bei über 60.000 gespeicherten Erkennungen lediglich rund 3,4 MB. Bei diesen Größenordnungen bringt eine inkrementelle Strategie keinen nennenswerten Vorteil, würde jedoch die Komplexität der Wiederherstellung erhöhen, da inkrementelle Backups stets auf einem vollständigen Basis-Backup aufbauen und in korrekter Reihenfolge eingespielt werden müssen.

### 6.5.2 Manuelles Backup und Wiederherstellung

Zusätzlich zu den automatisierten Backups steht ein manuelles Backup-Script zur Verfügung. Dieses startet einen temporären PostgreSQL-Container, der den Dump erstellt und anschließend sofort wieder entfernt wird. Ansonsten ist dieser funktionell gleich aufgebaut, wie das oben aufgezeigte System für automatisierte Backups.

Für die Wiederherstellung existiert ebenfalls ein Script, welches einen vollständigen Disaster-Recovery-Prozess implementiert:

Das Script fordert zunächst eine explizite Bestätigung, da alle in der Datenbank bestehenden Daten überschrieben werden müssen. Anschließend werden alle aktiven Datenbankverbindungen terminiert, die bestehende Datenbank gelöscht und neu angelegt, bevor das komprimierte Backup mittels `pg_restore` [**postgres\_pgrestore**] eingespielt wird.

**Bild 23:** Restore Flow

Das Recovery Point Objective (RPO) entspricht dem konfigurierten Backup-Intervall (standardmäßig 24 Stunden), das Recovery Time Objective (RTO) beträgt einige Sekunden bis wenige Minuten und ist primär durch die Größe des Backups bestimmt.

## 6.6 Documentation as Code (MkDocs)

Die Entwicklerdokumentation wird im Repository gepflegt und automatisiert als statische Website veröffentlicht. Hierfür wird MkDocs [**mkdocs**] mit dem Material-Theme [**mkdocs\_material**] eingesetzt, ein Python-basierter Static-Site-Generator, welcher Markdown-Dateien in eine Dokumentationswebsite transformiert. Die Verwendung von MkDocs hat den Vorteil, dass out-of-the-box viele Grundfunktionen einer guten Dokumentation, wie etwa eine Navigationsleiste oder Suchfunktion, bereits als Module funktionsfähig bereitgestellt werden.

Die Dokumentation ist in sechs Hauptbereiche gegliedert:

- Getting Started Voraussetzungen, Installation und Konfiguration für neue Entwickler
- Development: Entwicklungsumgebung, Docker-Befehle und Service-Entwicklung
- Operations: Backup-Verfahren, Wiederherstellung und Monitoring
- Deployment: Übersicht, Image-Erstellung und Produktionseinrichtung
- Reference: Script-Referenzen, Docker-Compose-Dokumentation, API-Dokumentation und Data-Collection-Flow

Die Veröffentlichung von Änderungen erfolgt automatisiert über einen eigenen GitHub-Actions-Workflow. Im Falle eines Pushes wird die Dokumentation neu gebaut und auf GitHub Pages veröffentlicht. Dadurch ist die Dokumentation stets synchron mit dem aktuellen Stand des Codes und unter einer permanenten, öffentlichen URL erreichbar.

Dieser Ansatz folgt dem Documentation-as-Code-Prinzip, bedeutet, dass die Dokumentation derselben Versionskontrolle wie der Quellcode unterliegt und Änderungen denselben Review-Prozess (Pull Requests) durchlaufen. Dies hat in der Entwicklung den großen Vorteil, dass die Dokumentation als Teil der Code-Basis gesehen und verwaltet wird und so einfacher und schneller bei Änderungen angepasst werden kann.

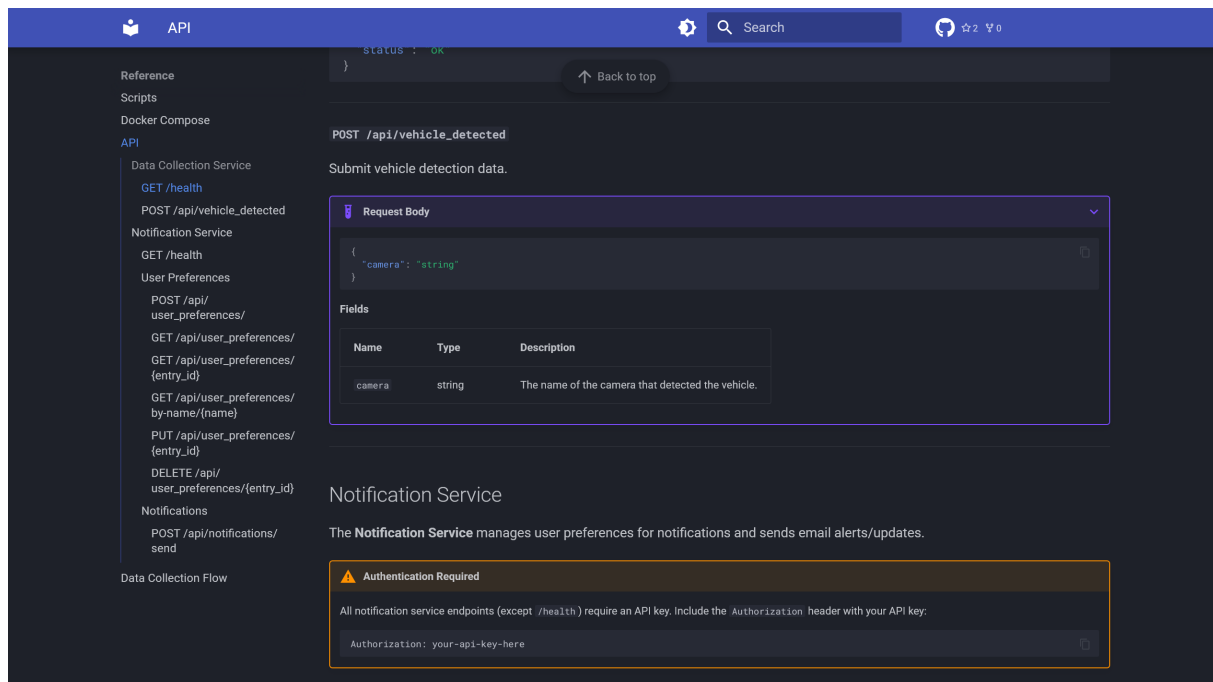


Bild 24: Entwicklerdokumentation mittels MkDocs

## 7 Qualitätssicherung und Tests

Das folgende Kapitel beschreibt die Maßnahmen zur Sicherstellung der Softwarequalität sowie die Qualität des Gesamtsystems unter realen Einsatzbedingungen.

### 7.1 Teststrategie

Die Teststrategie des Projekts kombiniert Unit-Tests zur Überprüfung einzelner Komponenten mit Integrationstests, welche das Zusammenspiel der Services auf API-Ebene validieren. Als Test-Framework wird Pytest **[pytest]** eingesetzt, ergänzt durch das Pytest-Coverage Modul **[pytest\_cov]** für Codeabdeckungsanalysen. Hierbei wurde das Ziel festgelegt, eine Testabdeckung von über 90% für jeden Python-Service zu erreichen.

#### 7.1.1 Unit-Tests

Die Unit-Tests decken die Handler-Klassen beider Python-Services ab und sind nach dem Arrange-Act-Assert-Pattern (AAA) strukturiert. Externe Abhängigkeiten wie die Synology-API, Plate Recognizer und der SMTP-Server werden durch die Mocking-Library `unittest.mock` simuliert, um die Tests unabhängig von externen Services ausführbar zu machen. Im Data Collection Service werden alle vier Handler-Klassen einzeln getestet:

- **CameraHandler:** Tests, welche die Interaktion mit der Synology-API durch gemockte HTTP-Responses simulieren. Dies umfasst Erfolgs- und Fehlerszenarien für Authentifizierung, Kameraabruf und Snapshot-Erstellung.
- **PlateRecognizerHandler:** Tests für den ALPR-API-Aufruf, geprüft mit gemockten HTTP-Responses für erfolgreiche und fehlgeschlagene Anfragen.
- **CountryHandler:** Die Bezirkserkennung für österreichische und slowenische Kennzeichen mit verschiedenen Eingabekombinationen wird mit Tests überprüft (z.B. unbekannter Ländercode, ein- und zweibuchstabile Kürzel).
- **DatabaseHandler:** Tests für Datenextraktion aus API-Antworten, Hash-Normalisierung (Groß-/Kleinschreibung, Leerzeichen) und die zeitenfensterbasierte Duplikatserkennung mit Datenbankinteraktionen.

Im Notification Service werden ebenfalls die Handler und Datenbankschicht getestet:

- **EmailHandler:** Tests für den SMTP-Versand mit gemocktem SMTP-Server, Bulk-Versand mit teilweisen Fehlern sowie der korrekte Versand von Alert- und Update-E-Mails.
- **DatabaseHandler:** CRUD-Operationen für Benutzerpräferenzen, einschließlich Duplikat-E-Mail-Erkennung und Filterung nach Benachrichtigungspräferenzen.

### 7.1.2 Integrationstests

Die Integrationstests überprüfen die vollständigen API-Endpunkte unter Verwendung von FastAPI. Dieses ermöglicht das Senden von HTTP-Requests gegen die Applikation, ohne einen externen Server starten zu müssen.

Für den Notification Service umfasst dies die vollständige CRUD-Schnittstelle der Benutzerpräferenzen (Create, Read, Update, Delete) sowie die Benachrichtigungs-API mit verschiedenen Empfänger-Szenarien (alle Abonnenten, spezifische Empfänger, keine Empfänger, Zustellfehler).

### 7.1.3 Testisolation und Datenbankstrategie

Die Tests werden innerhalb der CI-Pipeline mit den zugehörigen Service-Containern gegen den PostgreSQL-Service ausgeführt, anstatt eine In-Memory-Datenbank wie SQLite als Ersatz zu verwenden.

Diese Entscheidung wurde getroffen, da SQLite wesentliche benutzte PostgreSQL-Funktionalitäten nicht identisch abbilden kann. Beispiele hierfür sind die Verwendung des nativen PostgreSQL-Enum-Datentyps `[postgresql_enum]` für die Fahrzeugorientierung sowie die Verwendung von zeitzonenbehafteten Zeitstempeln `[postgresql_datetime]`. SQLite bietet für diese Typen keine native Unterstützung `[sqlite_datatypes]`. Durch die Nutzung derselben Datenbank-Engine in den Tests wird sichergestellt, dass die Tests tatsächlich das Verhalten der Produktionsumgebung widerspiegeln und keine falsch-positiven Ergebnisse durch abweichendes Datenbankverhalten entstehen.

Beide Services verwenden ein identisches Muster zur Datenisolation in ihrer Pytest-Konfigurationsdatei:

**Listing 11:** Pytest-Fixture für Datenisolation

```
1 @pytest.fixture(scope='function')
2 def db():
3     """
4     Provides a SQLAlchemy session for each test function.
5     Each test will run within its own transaction, which is
6     then rolled back at the end of the test, ensuring data
7     isolation without dropping tables.
8     """
9     connection = test_engine.connect()
10    transaction = connection.begin()
11    session = TestingSessionLocal(bind=connection)
12
13    try:
14        yield session
15    finally:
16        session.close()
17        transaction.rollback()
18        connection.close()
```

Jeder Test wird innerhalb einer eigenen Datenbank-Transaktion ausgeführt, die nach dem Test automatisch zurückgerollt wird. Dadurch werden alle Datenbankänderungen eines Tests nach dessen Abschluss verworfen und die Tests beeinflussen sich gegenseitig nicht. Die Reihenfolge der Testausführung ist somit beliebig.

Für die Integrationstests wird zusätzlich die FastAPI Dependency-Injection überschrieben, sodass der Test-Client die Test-Datenbanksession und, im Fall des Notification Service, einen Test-API-Key verwendet.

## 7.2 Qualitätssicherung der Kennzeichenerkennung

Die Bewertung der Erkennungsrate in einem ALPR-System wird dadurch erschwert, dass eine exakte Dunkelziffer, also der Anteil der Fahrzeuge, welche das System passieren aber

nicht erkannt werden, ohne eine unabhängige Referenzmessung nicht ermittelt werden kann. Eine systematische Gegenüberstellung jeder einzelnen physischen Durchfahrt mit der entsprechenden Systemerkennung wurde im Rahmen dieser Arbeit nicht durchgeführt. Dies begründet sich dadurch, dass eine bloße Erfassung der Fahrzeuganzahl ohne erfolgreiche Kennzeichenidentifikation einen geringen operativen Zusatznutzen bieten würde, da die eigentliche Relevanz der Daten in der Analyse von Trends und statistischen Mittelwerten liegt.

Die folgenden Erkenntnisse basieren daher auf Beobachtungen während des Betriebszeitraums sowie auf der Analyse der gespeicherten Erkennungsdaten.

Mehrere Umgebungsfaktoren beeinflussen die Erkennungsqualität: Da die Zufahrt nach Süden ausgerichtet ist, ergeben sich bei tiefem Sonnenstand, insbesondere in den Morgen- und Abendstunden sowie im Winter, Gegenlichtbedingungen, welche die Kennzeichenlesbarkeit beeinträchtigen. Bei starkem Regen, Schneefall oder Nebel ist ebenfalls mit einer reduzierten Erkennungsrate zu rechnen, da die Sichtbarkeit der Kennzeichen physisch eingeschränkt ist. Nächtliche Erkennungen profitieren von der Infrarot-Beleuchtung der Kamera, die Kennzeichen auch bei Dunkelheit ausreichend beleuchtet.

Zur Sicherung der Datenqualität im Praxisbetrieb wurden zwei zentrale Strategien implementiert, welche bereits aus vorherigen Kapiteln bekannt sind:

Zunächst eliminiert die zeitfensterbasierte Duplikatserkennung Mehrfacherkennungen desselben Fahrzeugs bei langsamer Durchfahrt oder wiederholter Bewegung im Erkennungsbereich. Der konfigurierbare Intervall-Filter, wie in der Implementierung (Abschnitt 5.2.7) beschrieben, verhindert, dass ein Fahrzeug, das beispielsweise anhält und erneut anfährt, als separate Erkennung gezählt wird.

Weiters beschränkt die Regionseinschränkung bei der ALPR-Erkennung die Analyse auf die vier relevanten Länder (AT, HU, SI, DE). Dies verbessert die Genauigkeit, da das Modell länderspezifische Kennzeichenformate priorisiert und weniger Fehlinterpretationen bei der Zeichenerkennung auftreten.

## 8 Fazit und Ausblick

### 8.1 Zusammenfassung der Ergebnisse

Im Rahmen dieser Diplomarbeit wurde ein vollständiges System zur automatisierten Kennzeichenerkennung und Verkehrsanalyse für die Zotter Schokolade GmbH konzipiert, implementiert und in den Produktivbetrieb überführt. Dieses erfasst durchfahrende Fahrzeuge an der Betriebszufahrt mittels einer IP-Kamera, erkennt deren Kennzeichen über eine lokal betriebene ALPR-Lösung und speichert die anonymisierten Daten in einer PostgreSQL-Datenbank. Die gesammelten Daten werden über ein Grafana-Dashboard visualisiert.

Das System läuft seit dem 8. August 2025, mit Ausnahme von kleineren Ausfällen, durchgehend im Produktivbetrieb. Bis zur Verfassung dieser Dokumentation (Februar 2026) wurden über 60.000 Fahrzeugkennzeichen erfasst. Die dabei erzielten Erkennungsdaten sind als Richtwerte zu verstehen und nicht als absolute Kennzahlen, da eine vollständige Referenzmessung, also der Abgleich jeder einzelnen Durchfahrt mit der entsprechenden Erkennung, im Rahmen dieser Arbeit nicht durchgeführt wurde. Wie im Kapitel 7.2 erläutert, ist die Dunkelziffer nicht erfasster Fahrzeuge aus diesem Grund nicht exakt bezifferbar.

Die für diese Diplomarbeit gesetzten Ziele wurden wie in dieser Dokumentation direkt oder indirekt beschrieben nach bestem Wissen und Gewissen erfüllt.

Der Ansatz der hybriden Microservice-Architektur hat sich als passend für das Projekt erwiesen, da dieser eine klare Trennung der Verantwortlichkeiten ermöglicht und einzelne Komponenten unabhängig voneinander entwickelt werden konnten. Dennoch vereinfacht dieser Ansatz der zentralen Datenbank die Komplexität im Vergleich zu einer klassischen Microservice-Architektur erheblich. Ebenso bewährte sich die Wahl der Synology DVA-Serie als Hardware-Plattform, da sie sowohl die Kameraverwaltung als auch die Docker-basierte Applikation auf einem einzigen Gerät vereint. Die lokale Bildverarbeitung durch den Plate Recognizer SDK-Container gewährleistet, dass keinerlei Bilddaten das lokale Netzwerk verlassen, was den Datenschutzanforderungen vollständig entspricht.

### 8.2 Ausblick

Neben den offensichtlichen Erweiterungsmöglichkeiten wie der Fertigstellung der konzipierten, aber noch nicht implementierten Services, wurden im Laufe der Konzeptionsphase einige Vorschläge erwogen, welche zukünftig in das System integriert werden könnten:

- **Vollständige Implementierung des Web-Frontends:** Die Ablösung von Grafana durch die geplante Next.js-Applikation würde eine dedizierte Benutzeroberfläche bie-



ten, die nicht nur der Visualisierung dient, sondern auch die Verwaltung von Benachrichtigungseinstellungen und Systemkonfigurationen ermöglichen könnte.

- **Tiefere Hardware-Integration mit lokalen Modellen:** Aktuell wird die KI-Leistung der Synology DVA primär für die Fahrzeugerkennung genutzt, während das ALPR-Modell in einem separaten Container läuft. Zukünftig könnten KI-Modelle zur Kennzeichenerkennung direkt auf stärkerer Hardware laufen, um Kosten und Latenz zu minimieren.
- **Systematische Trefferquotenmessung:** Um die Verlässlichkeit der Daten statistisch zu untermauern, könnten Referenzmessungen durchgeführt werden. Hierbei würde über einen definierten Zeitraum die tatsächliche Anzahl der Durchfahrten erfasst und mit den Systemdaten abgeglichen werden, um die Dunkelziffer exakt zu bestimmen.
- **Erweiterung der regionalen Anreicherung:** Der Bezirkserkennungs-Anreicherung könnte um weitere Länder (z. B. Deutschland) erweitert werden, um auch für Fahrzeuge aus diesen Ländern eine genauere Herkunftsanalyse zu ermöglichen.
- **Prädiktive Analysen:** Mit der Implementierung des Analytics-Service könnten Modelle bereitgestellt werden, welche basierend auf historischen Daten das Verkehrsaufkommen für einen gewissen Zeitraum vorhersagen.
- **Active Directory Integration:** Die Umsetzung des Auth-Service zur Anbindung an das Active Directory des Unternehmens würde die Sicherheit und Benutzerverwaltung standardisieren.